Circular List

We have seen some video clippings restart from the beginning when the end is reached. There are music players which repeat the songs from the playlist. These are basically manifestations of the data structure called circular list.

As the name itself indicates, it is a list where the nodes form a cycle. We can loosely say that the last node is connected back to the first. At any given point in time, we will have a pointer to one of the nodes in the circular list.

It is also possible that the circular list could have two pointers in each node – one points to the next and the other points to the previous – thus forming a doubly linked circular list. These types of lists are extremely common in operating systems.

The interface of a circlular linked list is same as that of a singly linked list.

It supports the following operations.

- Initialize the list
- Deinitalize the list
- Insert before/after a node
- Delete the node following the current node in the list
- Traverse
- Search for a key

The structure of the circular list is same as that of a singly linked list.
head_ : this will point to one of the nodes. We may want to give some other name for it.
The node structure is same as the one we have used in singly linked list.

```
struct node
{
        int key_;
        struct node *next_;
};
typedef struct node node_t;


struct clist
{
        node_t *head_;
};
typedef struct clist clist_t;
```

- **Initialize the list**

  The head of the circular list is grounded.

- **Deinitalize the list**

  This requires a traversal of the routine and freeing of each node. We should always delete the node which succeeds the node to which we have a pointer. This is left to the reader as an exercise to implement it.

- **Insert**

  If the list is empty, make a new node – make it point itself and set the head_ of the list to point to this node.

  If there are nodes in the list, make the next of new node take the value of next of the node to which head_ points to, change the next of head_ to point to the new node created. Effectively the node would get inserted after the node to which head points to,

```
void insert(clist_t* ptr_clist, int key)

{

        node_t *temp = create_node(key);


   if (ptr_clist->head_ == NULL)

 {

      temp->next_ = temp;

      ptr_clist->head_ = temp;

   }

   else

 {

        temp->next_ = ptr_clist->head_->next_;

        ptr_clist->head_->next_ = temp;

   }

}
```

What would happen if we change the head to point to the new node added? I leave it to you to figure out.

- **Display**

  Normally, we do not prefer do-while loop. We prefer top tested loops than bottom tested loops. But, in case of circular lists, do-while loop is preferred. We start from the head_ node, go to the next one repeatedly until the head_ is reached again.

```
void disp_list(clist_t* ptr_clist)
{
   node_t *pres = ptr_clist->head_;

   if (pres == NULL) {
      printf("List is empty\n");
      return;
   }

   node_t* temp = pres;

   printf("Circular List: ");

   do {
      printf("%d ", temp->key_);
      temp = temp->next_;
   } while (temp != pres);

   printf("\n");
}
```

Josephus Problem:

This problem has been stated in literature in number of ways. This is one of the versions.

Josephus Problem asks you to find the last remaining survivor from n people standing in a circle. Counting begins at a specific person and moves around the circle in a fixed direction. In each step, k-1 people are skipped, and the k-th person is removed. The process repeats until only one person is left.

The steps involved in solving this problem are:

1. n people are numbered from 1 to n in order and are placed in a circle
2. Starting from a designated start point, we count to kth person.
3. The kth person is eliminated
4. Counting and elimination steps are repeated until a single person is left
5. Original number of that person is the solution.

Let's take a small example:

- **n = 7** people
- **k = 3** → eliminate every 3rd person

Step	**Counting**	**Eliminated**
1	1 → 2 → **3**	**3**
2	4 → 5 → **6**	**6**
3	7 → 1 → **2**	**2**
4	4 → 5 → **7**	**7**
5	1 → 4 → **5**	**5**
6	**1** → 4 → **1**	**1**
7	4	**4**

This is an interesting exercise to program.

Applications of circular list:

- Round Robin scheduling
- *Multiplayer Turn Management*
- *Music Playlist*
- *Image Slideshow*
- *ring-based algorithms and token passing*

Doubly Linked List

A doubly linked list is a list which allows us to travel to the next node or the previous node from the current node. To facilitate this, every node apart from the component has two pointers – one to the next normally called next and the other called prev for previous.

The interface of a doubly linked list is same as that of a singly linked list.

It supports the following operations.

- Initialize the list
- Deinitalize the list
- Insert anywhere in the list
- Delete anywhere in the list
- Traverse
- Search for a key

The doubly linked list structure will have two pointers:

head_ : this will point to the first node.

tail_ : this will point to the last node.

```
struct DLL {
        node_t *head_;
        node_t *tail_;
};
typedef struct DLL dll_t;
```

The node structure will also have two pointers – traditionally called prev and next.

```
struct node
{
   int key_;
   struct node *prev_;
   struct node *next_;
};
typedef struct node node_t;
```

- **Initialize the list**

  The head and the tail of the doubly linked list are grounded.

- **Deinitalize the list**

  This requires a traversal of the routine and freeing of each node. We should always have a pointer to the next node before freeing the current node. Otherwise, the remaining list will become garbage and therefore memory leak.

- **Insert anywhere in the list**

  In case of a singly linked list, we had to have two pointers, one following the other as we could not move back in the list. That restriction no more exists in a doubly linked list. To insert a node temp before the current node pres, we could do the following changes.

  temp->next_ = pres; temp->prev_ = pres->prev_;

  pres->prev_->next_ = temp; pres->prev_ = temp;

  You may observe that we make 4 changes while inserting.

  The above code works only if we are inserting in the middle. What if we insert in

  - An empty list : both head_ and tail_ of the DLL should be changed
  - In the beginning : head_ of the DLL has to change
  - At the end : tail_ of the DLL has to change

- **Delete anywhere in the list**

  We should consider the following cases.

  - Deletion in the middle
  - Deletion of the first node – head_ of DLL has to change
  - Deletion of the last node – tail_ of DLL has to change
  - Deletion of only one node – both head_ and tail_ of DLL have to change.

  What do you think this code will do if temp point a node in the middle of the list?

  temp->next_->prev_ = temp->prev_;

  temp->prev_->next_ = temp->next_;;

  free(temp);

- **Traverse** :

  We can traverse forward or backward as we have links in both the directions.

Why doubly linked lists?

A singly linked list can do whatever a doubly linked list can do. Then, what do we gain by creating a doubly linked list?

As we can traverse in both directions, insertion, deletion become easier.

We use more space – extra pointer in each node. We gain in time.

This is a classical example of **Time Space Trade Off.**

Let us compare the performance of singly and doubly linked lists in terms of number of steps required for each of the following operations in the worst case. Assume that the list has n nodes.

- Move forward : singly linked list : 1 step; doubly linked list : 1 step : no difference
- Move back : singly linked list : n step; doubly linked list : 1 step :
- Insert before a node : singly linked list : 1 step; doubly linked list : 1 step :
- Delete a node : singly linked list : 1 step; doubly linked list : 1 step :
- Searching for a key : singly linked list : n step; doubly linked list : n step :

A doubly linked list does NOT make searching faster.

The advantage is **bidirectional traversal and easier local insertion/deletion***.*

Applications of Doubly Linked List :

Browser History

Music Player Playlist

Undo and Redo operations

LRU cache

Round Robin Scheduling

Queue

So far, we have learnt the data structures list, stack, circular list. We have observed that these are all abstract data types supporting a well-defined interface. We have also observed that each of these abstract data types could have different implementations. These data structures are found to be useful in different applications based on their interfaces.

Consider the case of printing a few files on a printer. We can fire the print command on many files all at once. How would the printer handle this? Would the files get mixed up? Not at all. The files are printed in the order in which they arrive at the printer. Whichever comes first will be printed first. The printer follows "First Come First Served" principle. To support this behaviour, the files are put in a **queue**. The files are serviced from the **front** and are attached at the **rear**.

Queue is a data structure which has two ends – normally called the rear and the front. The components are added at the rear and removed from the front. We say that the queue exhibits the behaviour - FIFO – First In First Out.

Interface of the queue:

Apart from initialization and deinitialization, the queue supports the following operations.

initialize()	Create an empty queue
enqueue(x)	Insert x at the rear
dequeue()	Remove the element at the front
front() / peek()	Examine the front element
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations
destroy() / deinitialize()	Release resources

Implementation : We can implement the queue in different ways. We will discuss “implementing the queue using a list” in this section. We will discuss “implementing the queue using an array in the next section”.

The queue shall have the following structure. It has two pointers referred to as front_ and rear_.

```
struct queue
{
   node_t* front_;
   node_t* rear_;
};
typedef struct queue queue_t;
```

The node shall have the component and a pointer to a node – it is a classical self-referential structure.

```
struct node
{
   int key_;
   struct node* next_;
};
typedef struct node node_t;
```

For illustration purpose, the component type is chosen as an int.

Design consideration : How to link the nodes?

The two major operations on a queue are :

Adding at the rear : enqueue

Removing from the front : dequeue.

How to support these operations efficiently?

Design 1 :

One possible arrangement could be as follows.

Rear of the queue will point to the last node. The last node will point to the node which came before that. This in turn points to the node which came before that and so on. The front will point to the first node.

You will observe that enqueue will require a few operations. Create a node. Populate it. Make the pointer field take the value of rear of the queue. Change rear to point to the new node created.

So far so fine.

How about removing the first node from the front? This would require that the front should point to the node after the first node. But the pointer field in that node is grounded. As this is a singly linked data structure, we cannot move back. The only way we can satisfy this is by travelling all the way from the rear to the front. So, deque would take as many operations as the number of nodes in the queue. This is not acceptable.

Design 2 :

Let the node pointed to by the front of the queue point to the next node. Each node shall point to the node which came later. The rear of the queue shall point to the last node whose pointer field is grounded.

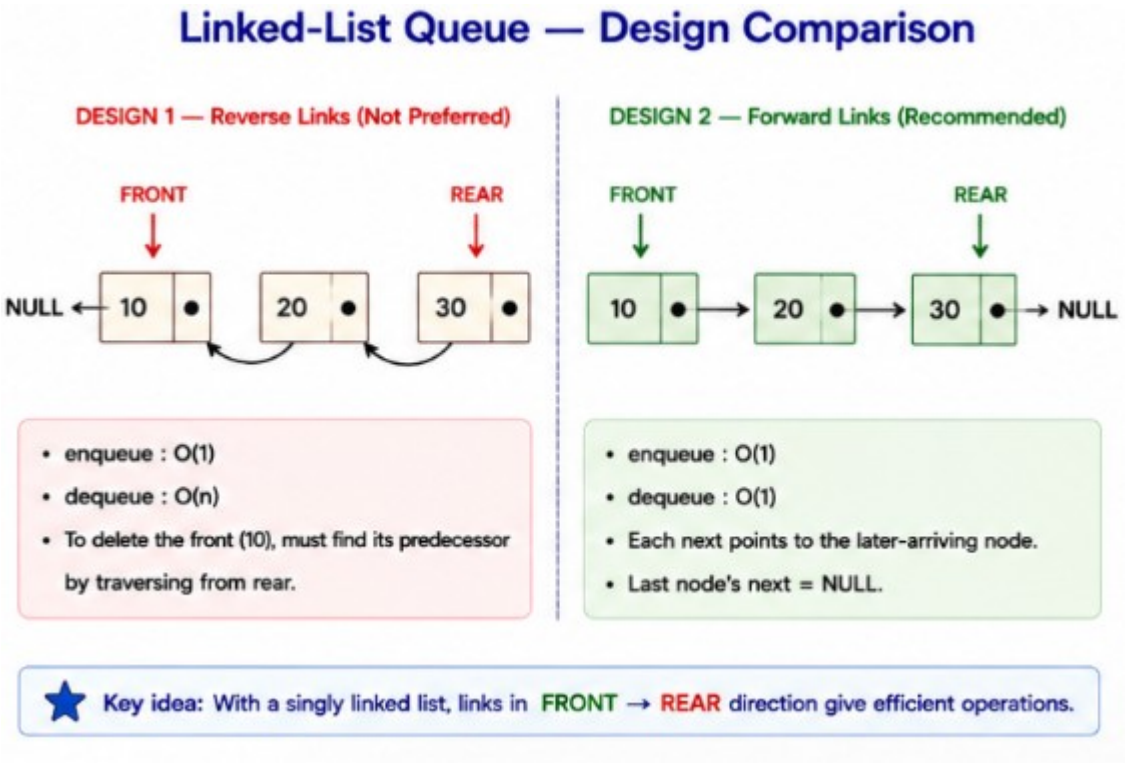
Enqueue requires that we create a node, populate it, change the pointer field of the node pointed to by rear to this new node, ground the pointer field in the new node, change rear to point to this new node.

This requires just a few operations. So fine.

Dequeue requires that we change the front to take the pointer in the first node so that it points to the next node.

This also requires just a few operations. So fine.

So, in out design each node will point to the node which came next.



Implementation:

- **Initialize the list**

  Both the front and the rear of the queue are initialized to NULL.

  ```
  void init_queue(queue_t *ptr_queue)
  {
      ptr_queue->rear_ = ptr_queue->front_ = NULL;
  }
  ```

- **Deinitalize the list**

  This requires a traversal of the routine and freeing of each node. We could start from the front. Make that the pres node. Store the next of pres in a pointer. Delete pres. Make pres take the stored pointer value. Repeat this until all nodes are deleted.

  Could we call deque until the queue becomes empty?

  ```
  void deinit_queue(queue_t *ptr_queue)
  {
      while(! is_empty(ptr_queue))
      {
              deque(ptr_queue);
      }
  ```

```
    ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

- **Enqueue**

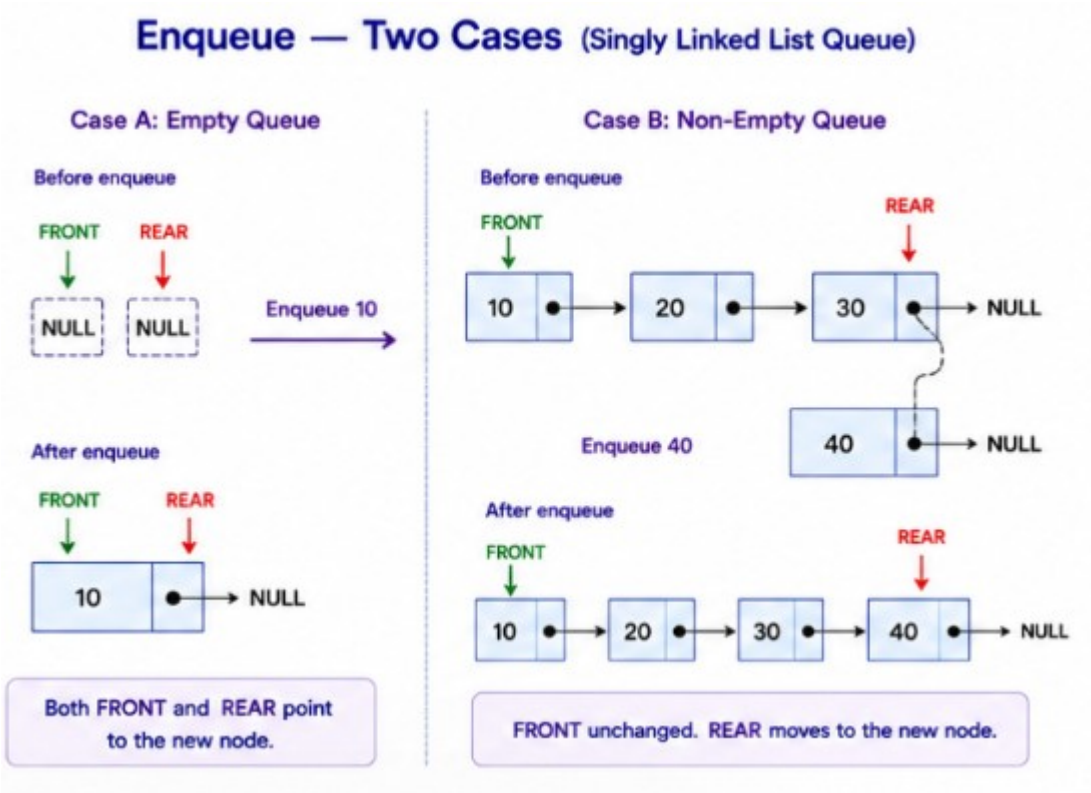
  If the malloc operation fails, it means that the queue is full. It is an unusual case – an exception. We do not know what the user wants to do. So, we will give a message to the user.

  We shall consider two cases .

  1. empty queue
     - create a node with the component.
     - Make rear of the queue point to it.
     - Ground the pointer in the new node created.
     - **Make front also point to the new node created**

       When there is only one node, it is both the front and the rear!

  2. queue with some nodes
     - create a node with the component.
     - Make the pointer in the node to which rear points to, to point to the new node created.
     - Make rear of the queue point to the new node.



```
void enque(queue_t *ptr_queue, int key)
{
    if(! is_full(ptr_queue))
    {
            node_t* temp = create_node(key);
            temp->next_ = NULL;
            if(ptr_queue->rear_ == NULL)
            {
                    ptr_queue->front_ = temp;
            }
            else
            {
                    ptr_queue->rear_->next_ = temp;
            }
            ptr_queue->rear_ = temp;
    }
    else
    {
            printf("queue full; cannot push\n");
    }
}
```

Create a node :

Enque calls this function to create a node. It is a good idea to develop a function for a particular purpose. This function is also not required for the client. By making the function, we could make this available in the file and not outside this file. Such a function is prefixed with the key static – which in this context means that the function is visible in this file alone – technically said to have internal linkage.

```
static node_t* create_node(int key)

{

        node_t* temp = (node_t*)malloc(sizeof(node_t));

        temp->key_ = key;

        return temp;

}
```

- **Dequeue** :

Which cases should we consider.

1. Empty queue : We cannot remove an element from an empty queue. We will give a message to the user.
2. Queue with some nodes.
   Please watch these steps. The order matters.
   - Copy the front of the queue to a local pointer variable say temp.
   - Copy the component in this node to a local variable say result
   - Change front to point to the next node by copying the pointer value in the first node
   - Remove the node to which temp points to.
   - Return the component stored in the variable result.

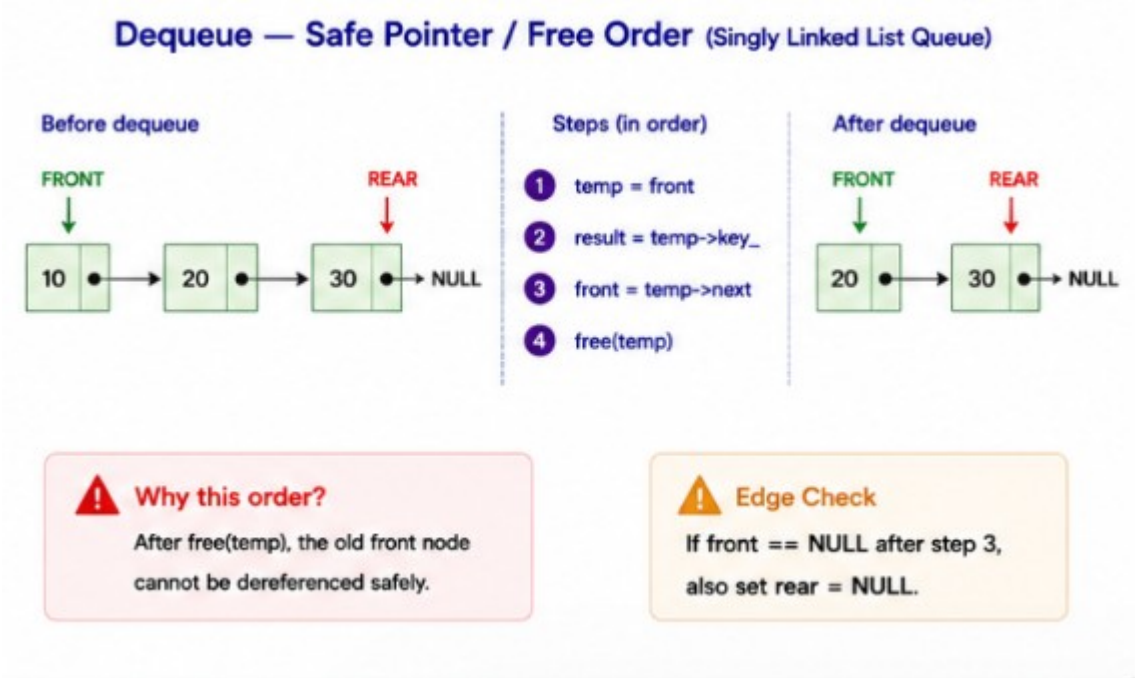
   It is interesting to think as to what happens if the order is modified!

   Why not free the node to which front points to?

   Why not just change the front?

   This procedure has a serious bug. Think of what happens if you are the only person in a queue. So you are the front and the rear. What if you are served? The front shall no more point to you. It is grounded. What about the rear?

   When there is only one node in the queue and it is dequeued, then the rear also will change!



```
int deque(queue_t *ptr_queue)
{
	if(! is_empty(ptr_queue))
	{
		int key;  node_t* temp = ptr_queue->front_;
		key = temp->key_;

		ptr_queue->front_ = temp->next_;
		if(ptr_queue->front_ == NULL)
		{
			ptr_queue->rear_ = NULL;
		}
		free(temp);
		return key;
	}
	else
	{
		printf("queue empty; cannot pop\n");
		return '\0';
	}
}
```

Dequeue is returning 0 when it fails. It is possible in our case as the component is integer. We could avoid returning any value from deque.

- **is_empty :**

  The queue is empty if the front is grounded. Can we say the same if rear is grounded?

  ```
  int is_empty(queue_t *ptr_queue)

  {

      return ptr_queue->front_ == NULL;

  }
  ```

- **Is_full**:
  This is tricky. Unless malloc fails, we will not know whether queue is full. If we call malloc and it fails, it is a bit too late to signal that the queue is full. We try to allocate. If it succeeds, we will conclude that the queue is not full. We free the memory allocated as it is not required now and is allocated just to check whether the allocation succeeds.

  ```
  int is_full(queue_t *ptr_queue)
  {
     node_t* temp;
       temp = (node_t*)malloc(sizeof(node_t));
     if (temp == NULL) {
        return 1;
     }
     free(temp);
     return 0;
  }
  ```

  The client could use a menu driven program to experiment with enqueue and dequeue in various permutations.

```
printf("opt : 1 : enque; 2 : deque ; 0 : quit\n");
scanf("%d", &opt);
while(opt)
{
        switch(opt)
        {
                case 1 : scanf("%d", &val);
                                enque(&q, val);
                                break;
                case 2 : val = deque(&q);
                                if(val != 0)
                                        printf("deque %d\n", val);
                                break;
        }
        printf("opt : 1 : enque; 2 : deque ; 0 : quit\n");
        scanf("%d", &opt);
}
```

Observe that there is an input before the loop and input at the end of the loop. This is a characteristic of any menu driven program. We could avoid the duplication of the code by delegating the work to a function.

Applications of queue :

- CPU scheduling : processes in batch processing usually wait in a queue
- Printer Spooler : collects the jobs for printing in a queue
- Breadth First search : Looking for a key in a tree structure
- Simulation models : queues to simulate random events in queueing theory
- Message queues : sharing information in operating systems
- Toll booth : waiting of vehicles to pass through a toll booth

Queue

So far, we have studied the queue abstract data structure and its interface. We have also learnt how to implement the queue using a linked list.

Keeping the interface same, can we implement the queue using an array?

Let us examine the design.

Interface of the queue:

Apart from initialization and deinitialization, the queue supports the following operations.

initialize()	Create an empty queue
enqueue(x)	Insert x at the rear
dequeue()	Remove the element at the front
front() / peek()	Examine the front element
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations
destroy() / deinitialize()	Release resources

Implementation : We are implementing the queue using an array. A queue has to have two ends the front and the rear. The queue would contain an array of components and two indices the front and the rear.

```
#define MAXSIZE 5

struct queue

{

        int key_[MAXSIZE];

        int front_;

        int rear_;

};

typedef struct queue queue_t;
```

For illustration purpose, the component type is chosen as an int.

Design consideration : How to handle the indices front and rear in enqueue and dequeue?

a) Incrementing indices correctly

Initially both these could be same – some valid index. Could be zero. On enqueue we should increment the rear and deque we should increment the front.

The valid indices can be anything in between 0 and MAXSIZE – 1. If we keep incrementing, the index can go outside the array boundary. Or if we were to say that the queue is full when the rear reaches MAXSIZE, it could be wrong if there have been a few deque operations.

To avoid the index going outside the array bounds and to avoid saying that the queue is full when some elements have been freed by deque, we will use **modulo arithmetic**.

To increment the index say i, we shall say

i = (i + 1) % MAXSIZE

b) Checking for queue empty and queue full

If the rear catches the front, queue is full.
If the front catches the rear, queue is empty.
These could be fairly complicated conditions.
Can we make that simple by using the same condition for both queue full and queue empty?
We can provided we waste one location.
But this might be checked at different points in time enqueue and dequeue.
We shall discuss at length in these operations.

Implementation:

- **Initialize the list**

  Both the front and the rear of the queue are initialized to any valid index.

  We are initializing it to 0.

```
void init_queue(queue_t *ptr_queue)

{

    // could be any valid index in the array

    ptr_queue->rear_ = ptr_queue->front_ = 0;

}
```

- **Deinitalize the list**

  As there are no resources allocated while adding to the queue, resetting the rear and the front to any valid index will be sufficient.

```
void deinit_queue(queue_t *ptr_queue)

{

    ptr_queue->rear_ = ptr_queue->front_ = 0;

}
```

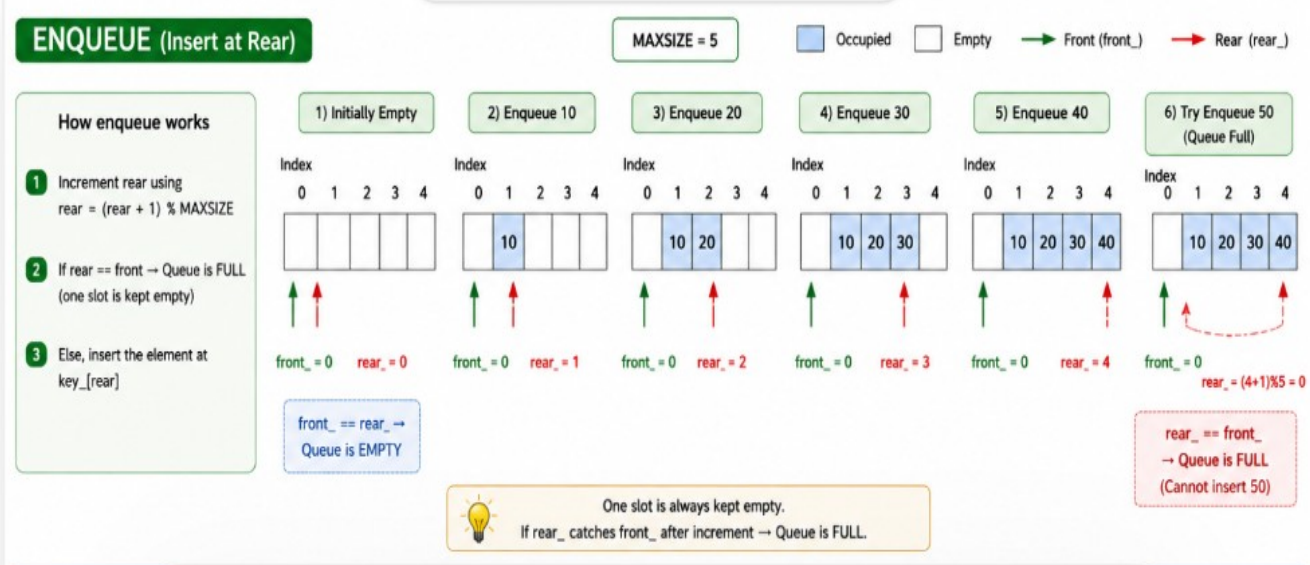
- **Enqueue**

  If the rear on incrementing using modular arithmetic catches the front, the queue is full.

  Note that we first increment the rear and compare with the front for checking for queue being full.

  If the queue is not full, use rear as the index and assign the component to the array.

  If the queue is full, reverse the change to rear.




```
void enque(queue_t *ptr_queue, int key)
{
    int temp;
    temp = ++ptr_queue->rear_ % MAXSIZE;
    ptr_queue->rear_ = temp;
    if(! is_full(ptr_queue))
    {
            ptr_queue->key_[temp] = key;
    }
    else
    {
                ptr_queue->rear_ = (ptr_queue->rear_ - 1 + MAXSIZE) % MAXSIZE;
                printf("queue full; cannot push\n");
    }
}
```

Should we take care of queue being empty as a special case?

==========

- **Dequeue** :

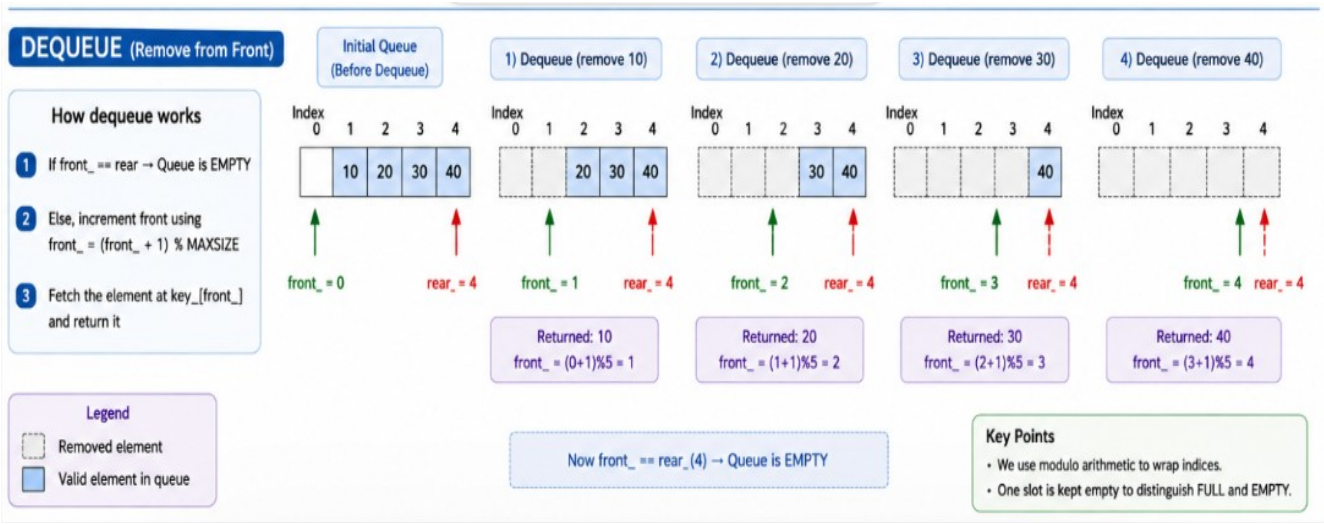
  We cannot deque if queue is empty. Remember the initialization of the queue. Both the front and the rear had the same value.

  We are wasting one location to make the condition checking for both queue full and queue empty same.

  If the front and the rear are equal before any incrementing, queue is empty.

If the front and the rear are equal after incrementing rear, queue is full.

If the queue is not empty, increment front using modulo arithmetic, use that the index and fetch the component.




```
int deque(queue_t *ptr_queue)
{
    if(! is_empty(ptr_queue))
    {
            int key;  int temp = ++ptr_queue->front_ % MAXSIZE;
            key = ptr_queue->key_[temp];
            ptr_queue->front_ = temp;
            return key;
    }
    else
    {
            printf("queue empty; cannot pop\n");
            return '\0';
    }
}
```

- **is_empty & is_full**

The condition is same in both – but at what time it is called in enqueue and deque differs.
To check queue is empty, deque calls this function before incrementing front.
To check queue is full, enque calls this function after incrementing rear.

```
// to be called before deque before incrementing

int is_empty(queue_t *ptr_queue)

{

    return ptr_queue->front_ == ptr_queue->rear_;

}


// to be called in enque after incrementing

int is_full(queue_t *ptr_queue)

{

    return ptr_queue->front_ == ptr_queue->rear_;

}
```

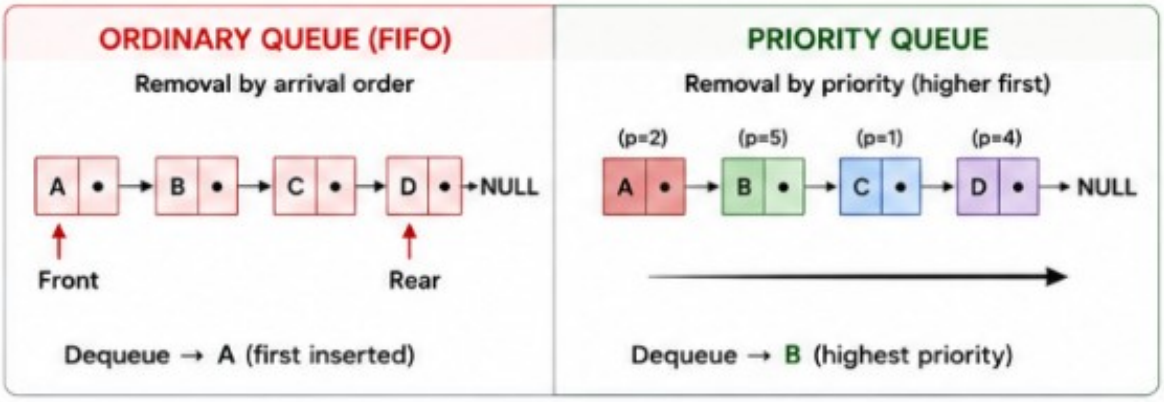
As expected, and required, the client code remains unaffected.

Priority Queue

So far, we have learnt the data structures list, stack, circular list and queue. We have observed that these are all abstract data types supporting a well-defined interface. We have also observed that each of these abstract data types could have different implementations. These data structures are found to be useful in different applications based on their interfaces.

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.



A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority. In Unix, the latter concept is followed. You may know that there is a command called nice using which you can reduce the priority of your process – you can be nice to others!

In our examples, we will assume that higher the numeric value, higher the priority.

What is the rationale behind this? Let us consider a couple of examples.

The patients wait in a queue to consult a doctor. But what if a patient becomes critically ill? Should not that patient get immediate attention?

Several processes could be running on an operating system. A few of them may have to respond in real time. Should not these get preference over others?

Our temples have an unusual way of implementing the priority queue. If you ready to pay more, you will get more priority!

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full

Implementation of priority queue:

A priority queue is a collection of elements in which every element has an associated priority, and deletion returns an element having the appropriate extreme priority.

A priority is an abstract data type. As we know, an **Abstract Data Type (ADT)** specifies **what operations are supported and what they mean**, without specifying how those operations are implemented.

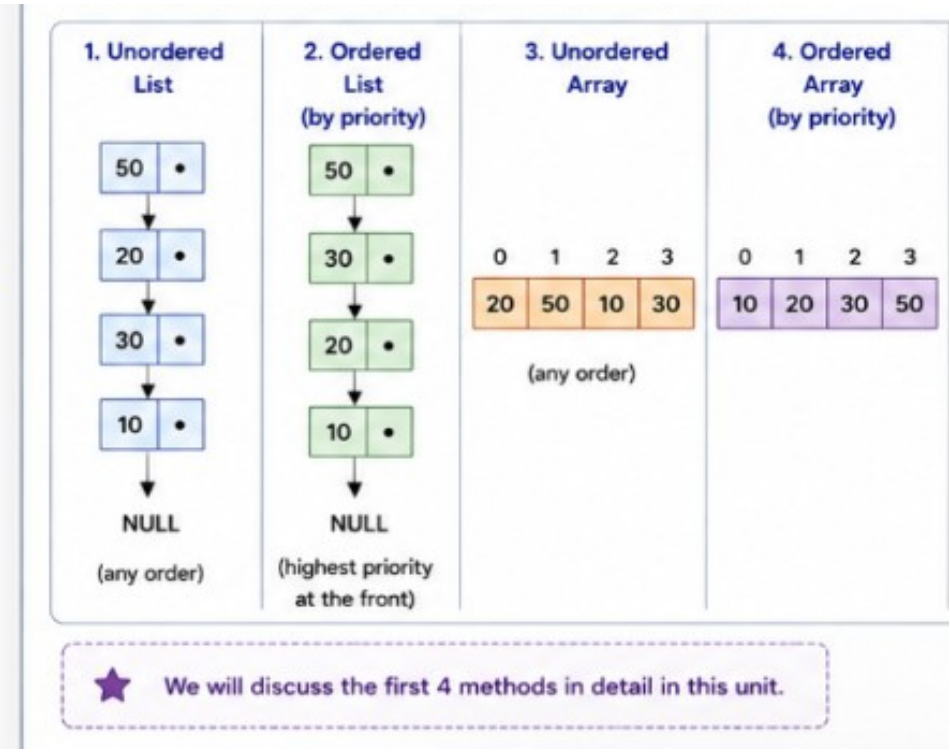
A priority queue can be implemented using any suitable underlying data structure. We will discuss the first 4 of the 5 listed below. The last one shall be discussed later in this course.

Different ways in which we may implement the priority queue:

1. Unodered list
2. Ordered list : ordered based on priority
3. Unordered array
4. Ordered array : ordered based on priority
5. Heap tree

Please note that these are not the only ways of implementing the ADT priority queue.

We will discuss the first 4 ways of implementing the priority queue in detail in the next few sections.



Applications of priority queue :

- CPU Scheduling of processes
- Hospital Emergency Services
- Printer Scheduling
- Network Packet Scheduling
- Dijkstra's Shortest Path Algorithm
- Prim's Minimum Spanning Tree Algorithm
- Event-Driven Simulation
- Huffman Coding

Comparison of queue and priority queue:

Priority Queue vs Ordinary Queue

Feature	**Ordinary Queue**	**Priority Queue**
Ordering principle	FIFO	Priority
Important attribute	Arrival time	Priority
Insert	Usually at rear	Huffman Coding
Delete	From front	Highest/lowest priority
Example	Printer jobs	CPU scheduling
Typical implementation	Array / linked list	Heap / ordered or unordered list

Comparison of various implementations:

We will discuss these again after all 4 ways of implementation are completed.

Implementation	**Insert**	**Delete priority item**
Unordered list	A few operations	Number of operations Proportional to number of elements
Ordered list	Number of operations Proportional to number of elements	A few operations
Unordered Array	A few operations	Number of operations Proportional to number of elements
Ordered Array	Number of operations Proportional to number of elements	A few operations
Binary heap	Number of operations proportional to	Number of operations proportional to

Implementation	Insert	Delete priority item
	Log to base 2 of number of elements	Log to base 2 of number of elements

Implementation:

We shall discuss in this section whatever is common to all the four implementations. Implementation specific to a particular selected data structure shall be discussed separately.

Job : We have called each component of the priority queue as a job – a term commonly used in operating systems. The job structure has a unique number to identify the job and a priority to indicate how it should be handled. We are assuming that higher this number, higher the priority.

Each new job is given the job number in an increasing order. This job number will help us make out in which order the jobs have been services.

```
struct job

{

        int job_no_;

        int priority_;

};

typedef struct job job_t;
```

This ADT job supports setting a job, displaying the job and return the priority of a job when queried.

```
void set_job(job_t* ptr_job, int job_no, int priority);

void disp_job(job_t* ptr_job);

int get_priority(job_t* ptr_job);
```

The following is the implementation of this interface. The code is self explanatory.

```
void set_job(job_t* ptr_job, int job_no, int priority)
```

```
{
        ptr_job->job_no_ = job_no;
        ptr_job->priority_ = priority;
}

void disp_job(job_t* ptr_job)
{
        printf("job # : %d priority : %d\n",
                ptr_job->job_no_, ptr_job->priority_);
}

int get_priority(job_t* ptr_job)
{
        return ptr_job->priority_;
}
```

Client :

The client code is menu driven. It reads an option before a loop is entered and an option again at the end of the loop. This is a characteristic of menu driven programs.

We support options for enque, deque and exiting the loop.

enque: This code takes the priority as input, increments the job number and calls enque of the underlying data structure. If the priority queue is full, the program is exited.

```
case 1 :        scanf("%d", &priority);
                set_job(&jobx, job_no++, priority);
                printf("Adding : ");
                disp_job(&jobx);
                enque(&q, &jobx);
                break;
```

deque: This code gets the job with the highest priority from the priority queue and then displays its contents. If the queue is empty, the program is exited.

```
case 2 :        jobx = deque(&q);
                printf("removing : ");
                disp_job(&jobx);
                break;
```

The job and the client will not change when we select different implementations of the priority queue.

Priority Queue : unordered list implementation

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using unordered list.

Interface of the priority queue:

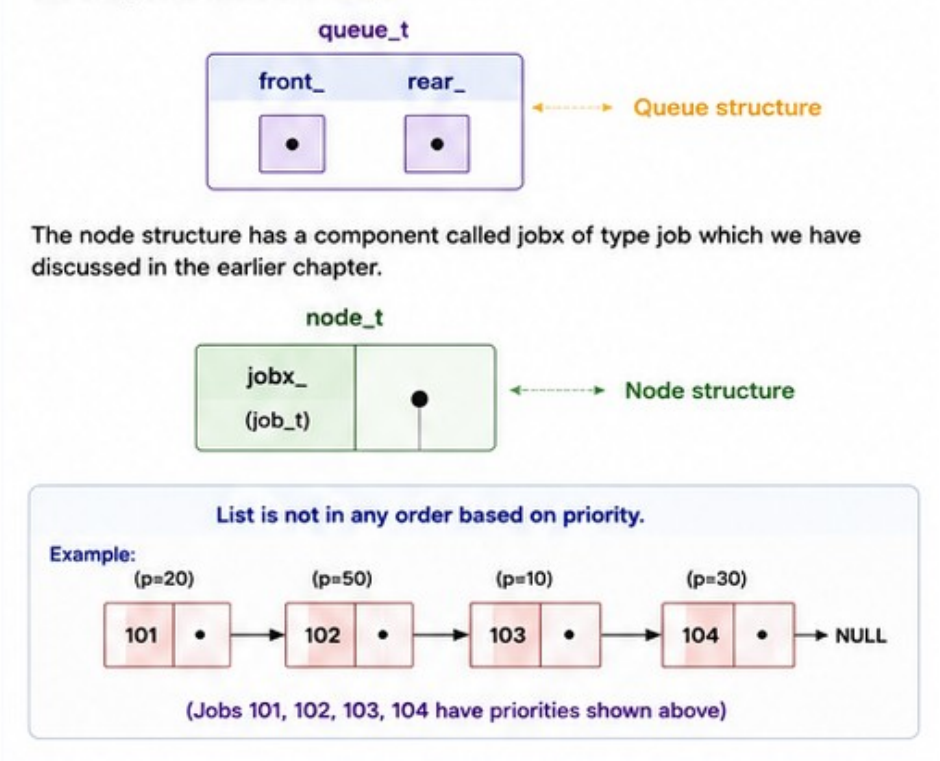
The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full

Implementation of priority queue:

The structure of the priority queue is same as that of queue using a linked list. It has two pointers to the node structure – one points to the beginning - called front and one points to the end – called the rear.

```
struct queue
{
        node_t* front_;
        node_t* rear_;
};
typedef struct queue queue_t;
```



The node structure has a component called jobx of type job which we have discussed in the earlier chapter.

```
struct node
{
        job_t jobx_;
        struct node* next_;
};
typedef struct node node_t;
```

The interface of priority queue remains same as that of queue. The implementation would dépend on the underlying data structure to support enque and deque appropriately.

```
void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enque(queue_t *ptr_queue, job_t *ptr_job);
job_t deque(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

The list is not in any order based on priority. Let us discuss each one of these operations.

Initialize:

Both the front and the rear of the queue are grounded.

```
 void init_queue(queue_t *ptr_queue)
{
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Deinitialize :

This is one of the ways of removing all the elements in the queue. Deque each element until the queue becomes empty. As deque is a costly operation, we could do it better. Can you try it out?

Look at this code below. Do you think this is required?

```
ptr_queue->rear_ = ptr_queue->front_ = NULL;
```

```
void deinit_queue(queue_t *ptr_queue)
{
        while(! is_empty(ptr_queue))
        {
                deque(ptr_queue);
        }
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Is full :

This is a trick. Try allocating. If it succeeds, then there is memory available. Release the memory allocated just for the testing purpose.

```
int is_full(queue_t *ptr_queue)
{
   node_t* temp;
  temp = (node_t*)malloc(sizeof(node_t));
   if (temp == NULL) {
      return 1;
   }
   free(temp);
   return 0;
}
```

Is empty:

Check whether the front of the queue is NULL.

Can we check the rear of the queue in stead?

Would front and rear being equal indicate that the queue is empty?

```
int is_empty(queue_t *ptr_queue)

{

        return ptr_queue->front_ == NULL;

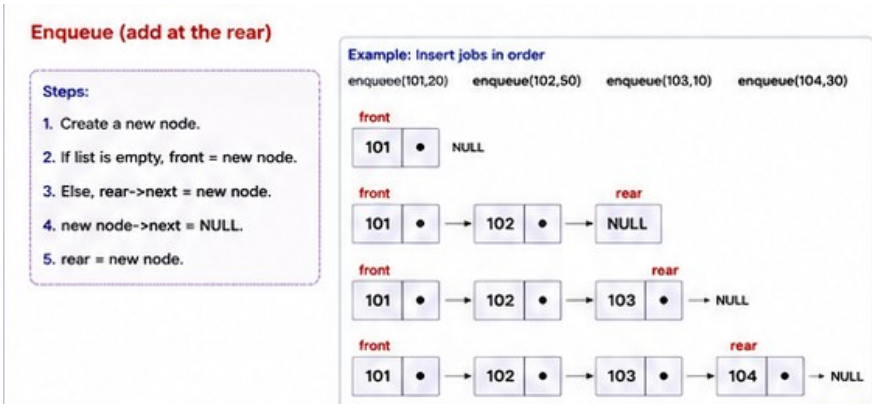
}
```

Enque :

This operation does not consider the priority as the list is unordered. It adds the element at the end.

If queue is full, the program is exited.

Make a node. Populate the node. Add at the rear. Change front if the queue is empty. Otherwise make next of rear the new node added. Ground the next of the new node. Change the rear to point to the new node.



```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{

	if(! is_full(ptr_queue))
	{
		node_t* temp = create_node(ptr_job);
		temp->next_ = NULL;
		if(ptr_queue->rear_ == NULL)
		{
			ptr_queue->front_ = temp;
		}
		else
		{
			ptr_queue->rear_->next_ = temp;
		}
		ptr_queue->rear_ = temp;
	}
	else
	{
		printf("queue full; cannot push\n");
	}
}
```

Deque :

We must find the node with the highest priority. If there are mode nodes with the same highest priority, we should select the job which came first. So, such a node will be closed to the front than the other nodes with the same priority.

If the queue is empty, the program is exited.

There are two parts in the algorithm.

The first is to find the position of the node with the maximum priority. We traverse the normal way.. We travel till the end of the list. We always keep track of the node with the highest priority – with two pointers – max_prev and max_pres.

The second part is to capture the job and then remove the node. If it is the first node, the front has to change. If it is the only node, the rear has to change. Otherwise we should make next of max_prev take the value of next of max_pres. Then release the max_pres node. Then return the job previously captured.

Do convince yourself that the order of operations matters.

A question for you – Can we remove the prev pointer from this algorithm?

```
job_t deque(queue_t *ptr_queue)

{

        if(! is_empty(ptr_queue))

        {

                // traverse

                node_t *prev = NULL;

                node_t *pres = ptr_queue->front_;

                int  max_priority = get_priority(&pres->jobx_);

                node_t *max_prev = NULL;

                node_t *max_pres = pres;

                int priority;

                while(pres != NULL)

                {

                        priority = get_priority(&pres->jobx_);
```

```
        if(priority > max_priority)
        {
                max_priority = priority;
                max_prev = prev;
                max_pres = pres;
        }

        prev = pres;
        pres = pres->next_;
}

job_t jobx;
jobx = max_pres->jobx_;

if(max_prev == NULL)
{
        ptr_queue->front_ = max_pres->next_;
        if(ptr_queue->front_ == NULL)
        {
                ptr_queue->rear_ = NULL;
        }
}
else
{
        max_prev->next_ = max_pres->next_;
        if(max_pres->next_ == NULL) // last node deleted
```

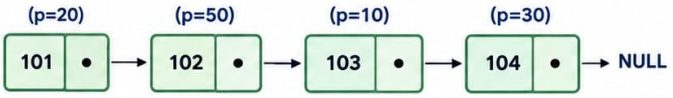
```
                {
                        ptr_queue->rear_ = max_prev;
                }
        }
        free(max_pres);
        return jobx;
    }
    else
    {
        printf("queue empty; cannot pop\n");
        exit(1);
    }
}
```

Dequeue (remove highest priority element)

Part 1: Find the node with highest priority

Keep track of the highest-priority node using two pointers: **max_prev** and **max_pres**.

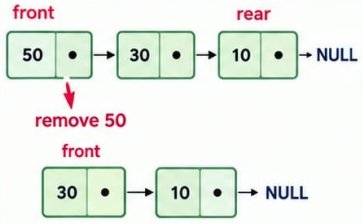
Example list



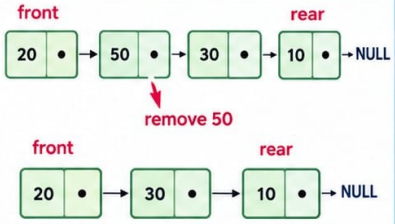
Step	pres (current)	max_prev	max_pres	Explanation
Start	101 (p=20)	NULL	101 (p=20)	Highest so far is 101
1	102 (p=50)	101	102 (p=50)	102 has higher priority
2	103 (p=10)	101	102 (p=50)	No change
3	104 (p=30)	101	102 (p=50)	No change

Part 2: Remove that node

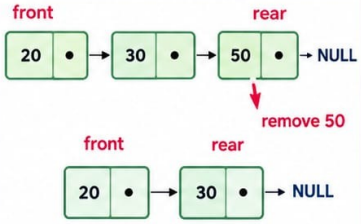
Case A: max_pres is front
(remove first node)



Case B: max_pres is in middle



Case C: max_pres is rear



Efficiency of operations :

Enque : This requires a few operations. We add at the rear without considering the priority.

Deque : This requires walking through the list to find the node with the highest priority. So, the number of operations is proportional to the number of nodes in the queue.

Priority Queue : ordered array implementation

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using an ordered array.

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations

1. **Implementation of priority queue: Using an ordered array**

We have discussed the implementation of priority queue using an unordered array. Now, we shall discuss the priority queue using an **ordered array**. The queue structure has an array of nodes. The node structure will remain the same. The interface will also remain the same!

We do not have rear and front. Instead, we have n, which is the count of number of nodes in the priority queue.

```
#define MAXSIZE 10
struct queue
{
        job_t key_[MAXSIZE];
        int n_; // # of elements
};
typedef struct queue queue_t;

struct node
{
        job_t jobx_;
        struct node* next_;
};
typedef struct node node_t;

void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enque(queue_t *ptr_queue, job_t *ptr_job);
job_t deque(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

The array is ordered based on priority. Let us discuss each one of these operations.

Initialize:

Make the number of elements in the priority queue zero.

```
void init_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Deinitialize :

There were no resources allocated during any of these processes dynamically.

Just change n to 0 indicating no elements in the priority queue.

```
void deinit_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Is full :

Check whether the number of elements is equal to the MAXSIZE of the array of nodes.

```
int is_full(queue_t *ptr_queue)
{
        return ptr_queue->n_ == MAXSIZE;
}
```

Is empty:

Check whether the number of elements in the priority queue is 0.

```
int is_empty(queue_t *ptr_queue)

{

        return ptr_queue->n_ == 0;

}
```

Enque :

We want to arrange the nodes such a way that the nodes with the highest priority are at the end of the array. We also want the nodes with some priority which came earlier to be towards higher indices compared to the ones which came later with the same priority.

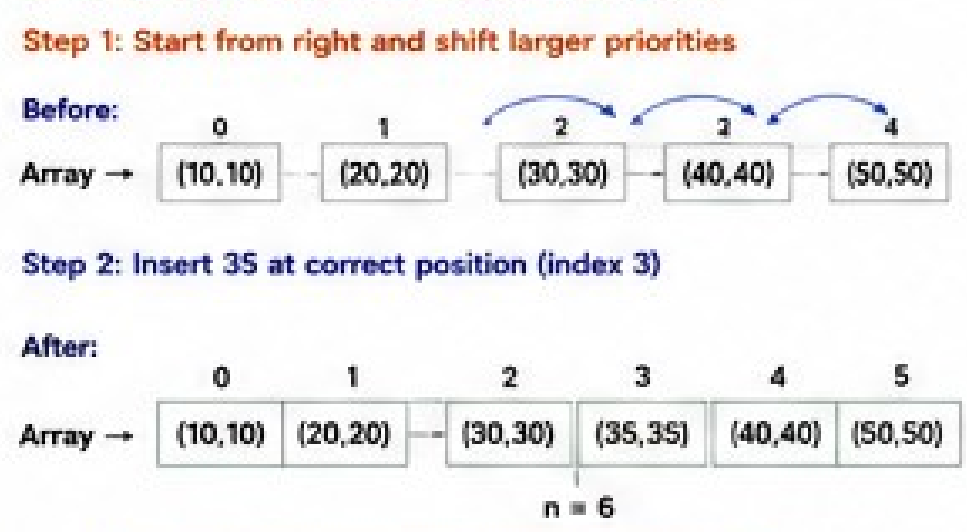
Enque is simple. Remove the last node in the array.

If the array is full, the program is exited.

Otherwise insert the new node at the right position. The nodes are being arranged in non-decreasing order of priority. We basically implement a very famous algorithm called insert. This is used in a sorting algorithm called insertionsort.

We traverse right to left and stop as soon as the right position is found to insert. As we know that we have to insert and make space for a new node, we keep shifting to the right every time we enter the loop.

Once we are out of the loop, we assign the new node to the right position.




```
// insert in order
```

```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{

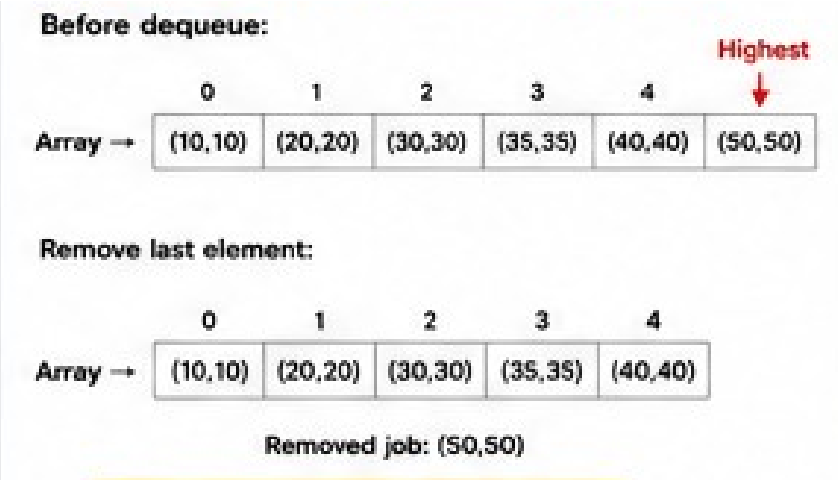
        if(! is_full(ptr_queue))
        {
                int n = ptr_queue->n_;
                int i = n - 1;
                int priority = get_priority(ptr_job);
                while(i >= 0 &&
                        get_priority(&ptr_queue->key_[i]) >= priority)
                {
                        ptr_queue->key_[i + 1] = ptr_queue->key_[i];
                        --i;

                }
                ptr_queue->key_[i + 1] = *ptr_job;
                ++ptr_queue->n_;
        }
        else
        {
                printf("queue full; cannot push\n");
                exit(1);
        }
}
```

Deque:

If the array is empty, the program is exited.

We always remove the last node.




```
// always the last one
job_t deque(queue_t *ptr_queue)
{
	if(! is_empty(ptr_queue))
	{
		job_t jobx = ptr_queue->key_[--ptr_queue->n_];
		return jobx;
	}
	else
	{
		printf("queue empty; cannot pop\n");
		exit(1);
	}
}
```

Efficiency of operations :

.

Deque: This requires a few operations. We remove the end as the last node will have the highest priority.

enque: This requires walking through the list from right to left to find the node with the highest priority. While finding the position, the elements shifted to the right. So, the number of operations in the worst case is proportional to the number of nodes in the queue.

Priority Queue : Different implementations

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using unordered list.

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations

1. **Implementation of priority queue: Using an Ordered List**

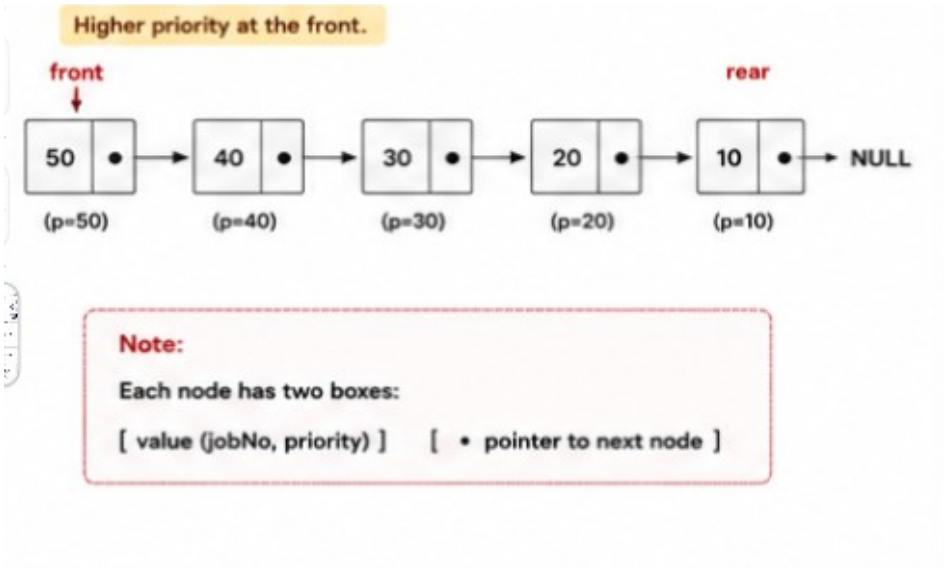
We have discussed the implementation of priority queue using an unordered list. Now, we shall discuss the priority queue using an ordered list. The queue structure will remain the same. The node structure will remain the same. The interface will also remain the same!

```
struct queue
{
        node_t* front_;
        node_t* rear_;
};
typedef struct queue queue_t;

struct node
{
        job_t jobx_;
        struct node* next_;
};
typedef struct node node_t;

void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enque(queue_t *ptr_queue, job_t *ptr_job);
job_t deque(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

The list is not in any order based on priority. Let us discuss each one of these operations.



Initialize:

No change. Both the front and the rear of the queue are grounded.

```
void init_queue(queue_t *ptr_queue)
{
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Deinitialize :

No change. This is one of the ways of removing all the elements in the queue. Deque each element until the queue becomes empty.

The following questions are still valid.

As deque is a costly operation, we could do it better. Can you try it out?

Look at this code below. Do you think this is required?

```
ptr_queue->rear_ = ptr_queue->front_ = NULL;
void deinit_queue(queue_t *ptr_queue)
{
        while(! is_empty(ptr_queue))
        {
                deque(ptr_queue);
        }
        ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

Is full :

No change. This is a trick. Try allocating. If it succeeds, then there is memory available. Release the memory allocated just for the testing purpose.

```
int is_full(queue_t *ptr_queue)
{
   node_t* temp;
  temp = (node_t*)malloc(sizeof(node_t));
   if (temp == NULL) {
      return 1;
   }
   free(temp);
   return 0;
}
```

Is empty:

No change. Check whether the front of the queue is NULL.

Can we check the rear of the queue in stead?

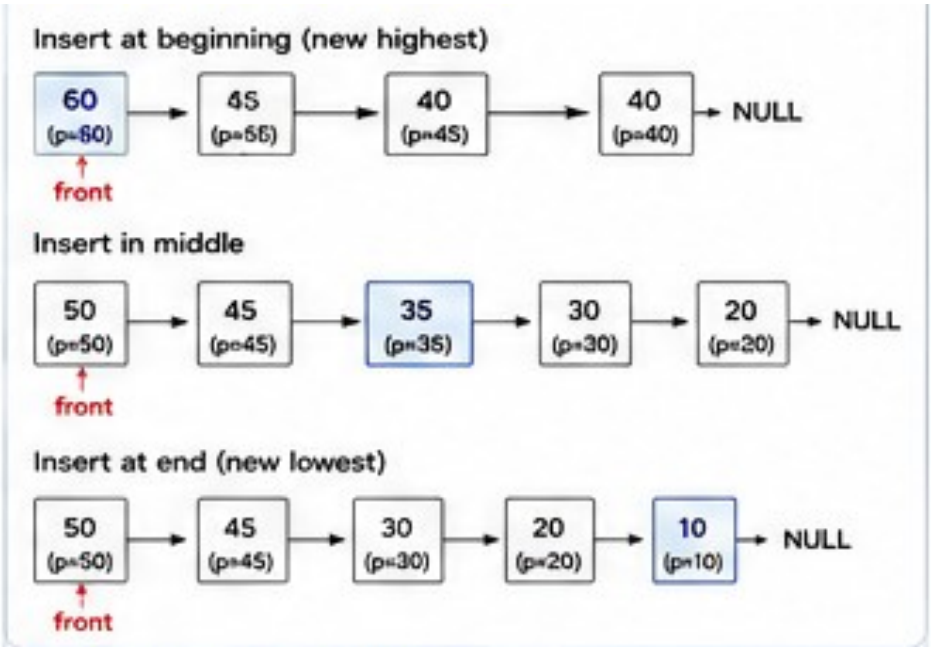
Would front and rear being equal indicate that the queue is empty?

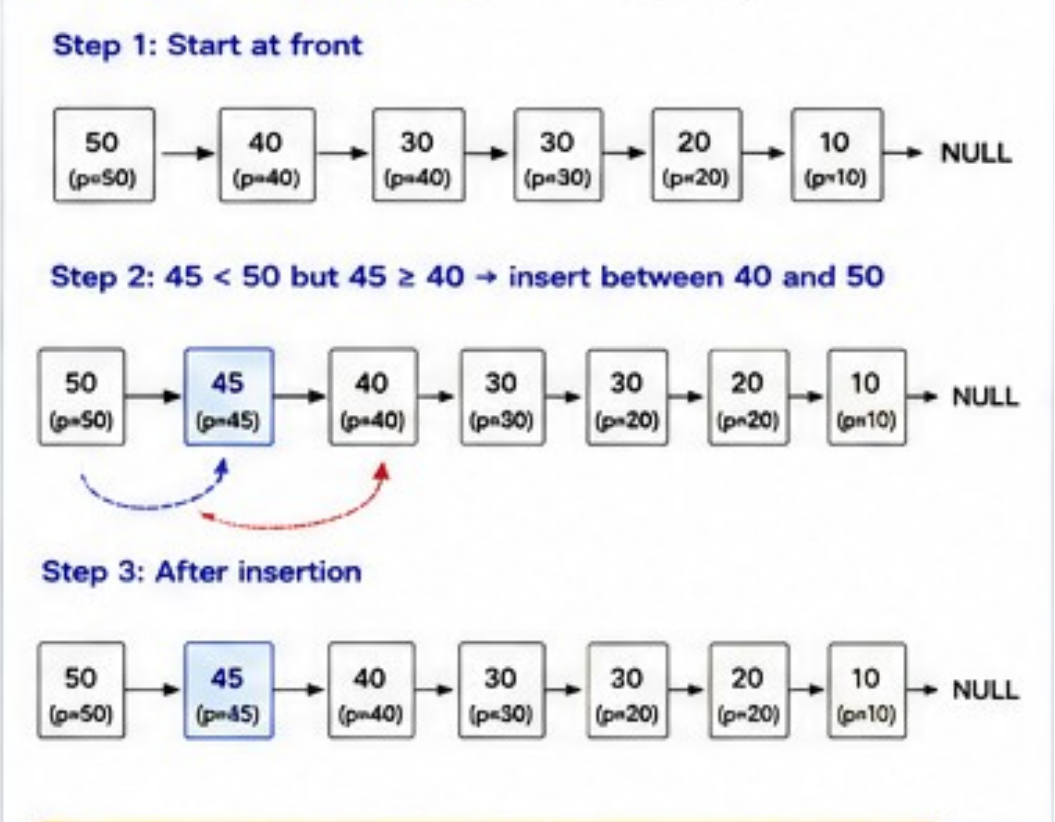
```
int is_empty(queue_t *ptr_queue)
{
        return ptr_queue->front_ == NULL;
}
```

Enque :

This operation does consider the priority while inserting the node.

If queue is full, the program is exited.





We will see to it that the node with the highest priority is always at the front.

What if more than one node has the same high priority. We will insert before the one with the same priority so that the newly added job gets services last amongst those with the same priority.

Make a node. Populate the node. Find the position to insert.

If the queue is empty, add at the rear. Also change the front to point to the new node created.

Otherwise traverse using two pointers prev and pres.

Compare the priority of the present node with that of the new node to decide whether we have found the position to insert.

Observe the relational operator >=. Would this work if the operator is changed to > ?

while(pres != NULL && get_priority(&pres->jobx_) >= get_priority(&temp->jobx_))

Once the location is found, check whether the insertion is in the beginning or the end or anywhere in between.

Make the changes in the queue as well as the new node added.

```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{
	if(! is_full(ptr_queue))
	{
		node_t* temp = create_node(ptr_job);
		// empty queue
		if(ptr_queue->rear_ == NULL)
		{
			ptr_queue->front_ = temp;
			ptr_queue->rear_ = temp;
			temp->next_ = NULL;
		}
		else // find the location to insert
		{
			node_t* prev = NULL; node_t* pres = ptr_queue->front_;
			while(pres != NULL &&
				get_priority(&pres->jobx_) >= get_priority(&temp->jobx_))
			{
				prev = pres;
				pres = pres->next_;
			}
			if(prev == NULL)
			{
				ptr_queue->front_ = temp;
			}
			else
```

```
                {
                        prev->next_ = temp;
                        if(pres == NULL)
                        {
                                ptr_queue->rear_ = temp;
                        }
                }
                temp->next_ = pres;
        }
    }
    else
    {
        printf("queue full; cannot push\n");
        exit(1);
    }
}
```

Deque :

As the priority queue is in an ordered list, the deque is straight forward. Remove the node to which front points to.

If the queue is empty, exit the program.

Otherwise, copy the pointer to the first node to a local variable.

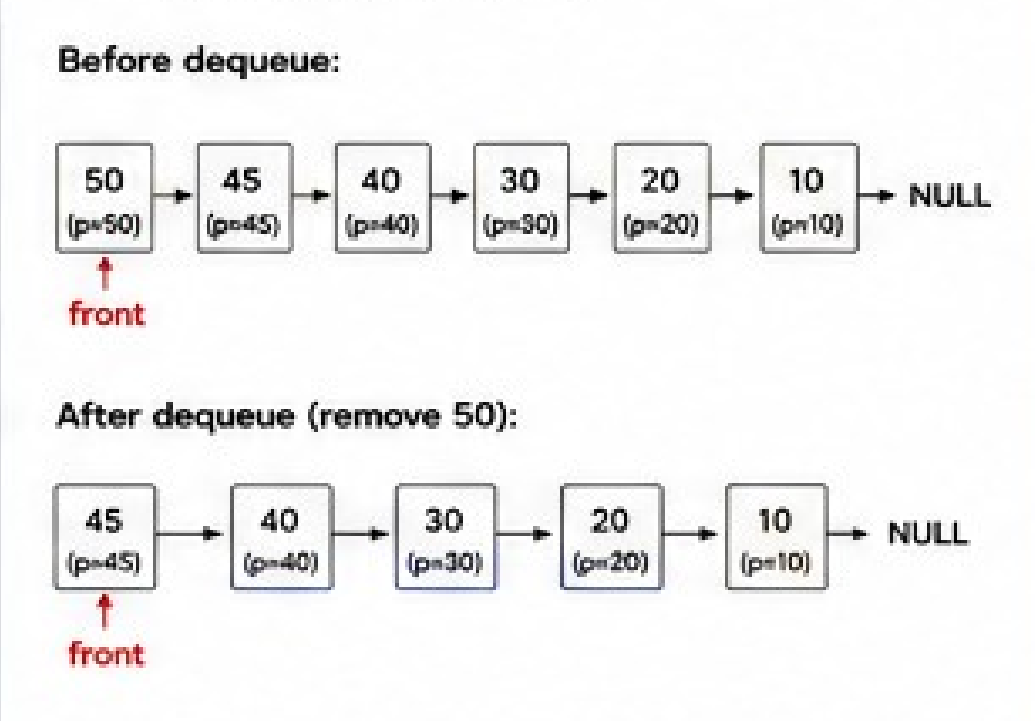
Copy the job to a local variable.

Change front to point to the node after the first.

If front becomes NULL, change the rear too.

Release the memory of the first node whose pointer has been copied.

Returned the job held in a local variable.




```
job_t deque(queue_t *ptr_queue)

{

        if(! is_empty(ptr_queue))

        {


                job_t jobx;  node_t* temp = ptr_queue->front_;

                jobx = temp->jobx_;


                ptr_queue->front_ = temp->next_;

                if(ptr_queue->front_ == NULL)

                {

                        ptr_queue->rear_ = NULL;

                }

                free(temp);
```

```
            return jobx;
        }
        else
        {
            printf("queue empty; cannot pop\n");
            exit(1);
        }
}
```

Efficiency of operations :

Enque : This requires walking through the list to find the node with the highest priority and then insert. So, the number of operations in the worst case is proportional to the number of nodes in the queue

Deque :. This requires a few operations. We remove from the front without considering the priority.

Priority Queue : unordered array implementation

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using unordered list.

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations

1. **Implementation of priority queue: Using an unordered array**

We have discussed the implementation of priority queue using an unordered list. Now, we shall discuss the priority queue using an **unordered array**. The queue structure is changed to have an array of nodes. The node structure will remain the same. The interface will also remain the same!

We do not have rear and front. Instead, we have n, which is the count of number of nodes in the priority queue.




```
#define MAXSIZE 10

struct queue

{

        job_t key_[MAXSIZE];

        int n_; // # of elements

};

typedef struct queue queue_t;


struct node

{

        job_t jobx_;

        struct node* next_;

};

typedef struct node node_t;


void init_queue(queue_t *ptr_queue);

void deinit_queue(queue_t *ptr_queue);

void enque(queue_t *ptr_queue, job_t *ptr_job);

job_t deque(queue_t *ptr_queue);
```

```
int is_empty(queue_t *ptr_queue);

int is_full(queue_t *ptr_queue);
```

The list is not in any order based on priority. Let us discuss each one of these operations.

Initialize:

Make the number of elements in the priority queue zero.

```
void init_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Deinitialize :

There were no resources allocated during any of these processes dynamically.

Just change n to 0 indicating no elements in the priority queue.

```
void deinit_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Is full :

Check whether the number of elements is equal to the MAXSIZE of the array of nodes.

```
int is_full(queue_t *ptr_queue)
{
        return ptr_queue->n_ == MAXSIZE;
}
```

Is empty:

Check whether the number of elements in the priority queue is 0.

```
int is_empty(queue_t *ptr_queue)

{

        return ptr_queue->n_ == 0;

}
```

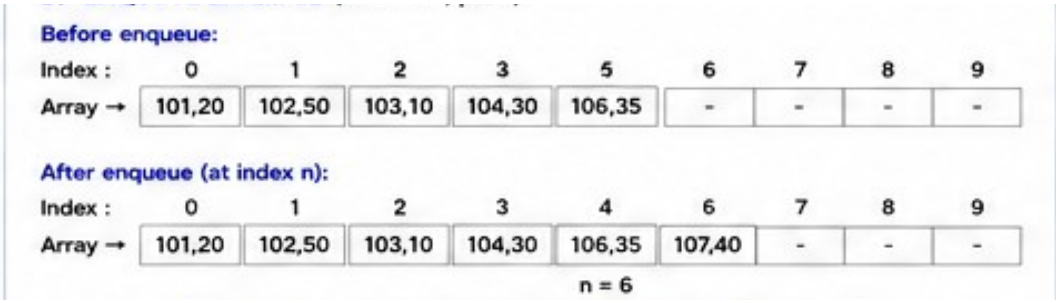
Enque :

As we have an unordered array, the enque is straight forward.

If the priority queue is full, the program is exited.

Otherwise use n as the index on the key – an array of nodes and assign the job to the array key at that location.

What if we get more than one node with the same priority. The one which came later will be added at a later position. So, while servicing the one which came earlier with the same priority will be catered to first. So, this is fine.



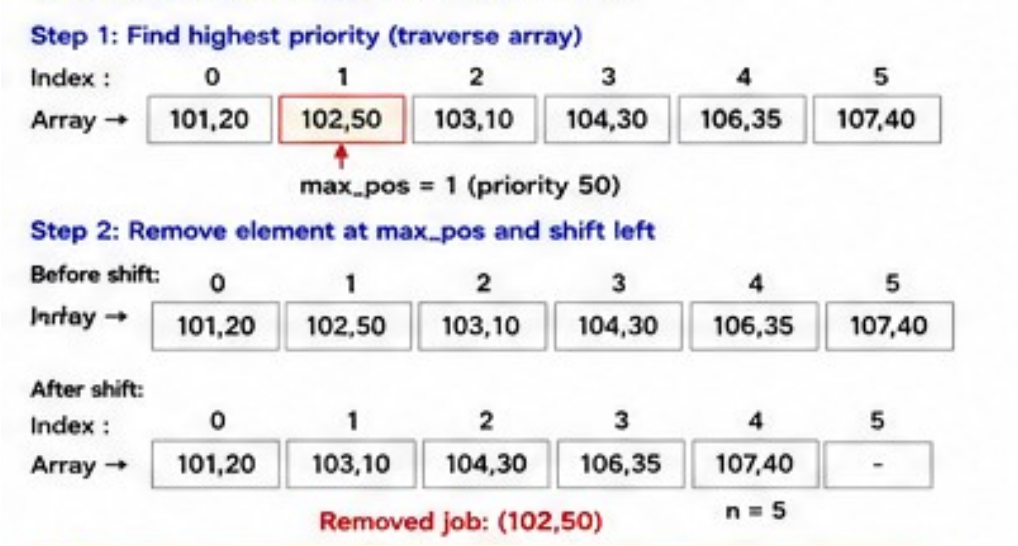
```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{
        if(! is_full(ptr_queue))
        {
                ptr_queue->key_[ptr_queue->n_++] = *ptr_job;
        }
        else
        {
                printf("queue full; cannot push\n");
                exit(1);
        }
}
```

Deque:

If the queue is empty, the program is exited.

There are two parts in this algorithm.

1. Find the node of the highest priority.
   It is calling finding the biggest in an array. We track the biggest so far in an index variable called max_pos. We have to traverse the whole array as there is no order of elements.
2. Remove that node.
   Once we find the position, we copy the job in that location to a local variable.
   How to remove an element from the array?
   We move the elements left by one position starting from the position where the max priority occurs. Then we decrease the number of elements in the priority queue.




```
job_t deque(queue_t *ptr_queue)
{
        if(! is_empty(ptr_queue))
        {
                int max_pos = 0;
                int n = ptr_queue->n_;
                for(int i = 1; i < n; ++i)
                {
                        if(get_priority(&ptr_queue->key_[i]) >
                                get_priority(&ptr_queue->key_[max_pos]))
                        {
                                max_pos = i;
                        }
                }
                job_t jobx = ptr_queue->key_[max_pos];
```

```
            // remove this
            for(int i = max_pos; i < n - 1; ++i)
            {
                ptr_queue->key_[i] = ptr_queue->key_[i+1];
            }
            --ptr_queue->n_;
            return jobx;
        }
        else
        {
            printf("queue empty; cannot pop\n");
            exit(1);
        }
}
```

Efficiency of operations :

.

Enque : This requires a few operations. We add at the end without considering the priority

Deque : This requires walking through the list to find the node with the highest priority and then remove it by shifting the elements from right to left. So, the number of operations is proportional to the number of nodes in the queue.

Tree

A few points:

So far in this course, we have discussed that a data structure has number of components. Each data structure is a type – called Abstract Data Type and each data structure supports a set of operations meaning for that data structure.

Each component knows about itself very strongly. It is cohesive. It does not know in which data structure it will be put.

Each data structure knows little about the component it contains. Whenever it requires any services from the component, it will delegate the function to that of the component. It does not probe inside the component to see what it contains. So, the data structure is also cohesive.

There is not much of dependency between the data structure and the component. They are not coupled.

By this way we realize one of the requirements of software development – high cohesion and low coupling.

We have discussed a few data structures like list, stack, queue, doubly linked list, circular queue. The components of all these data structures are in a line – they are linear. Even though the doubly linked list has two pointers in each node, they still are in a line. These data structures are called linear data structures.

There are many situations which cannot be captured in a line.

What If we trace father of a person, father's father and so on?

From Ramayana,

Lava's father is Rama. Rama father's is Dasharatha.

This is a linear relationship as a person has only one father.

What if we trace the relationship of children.

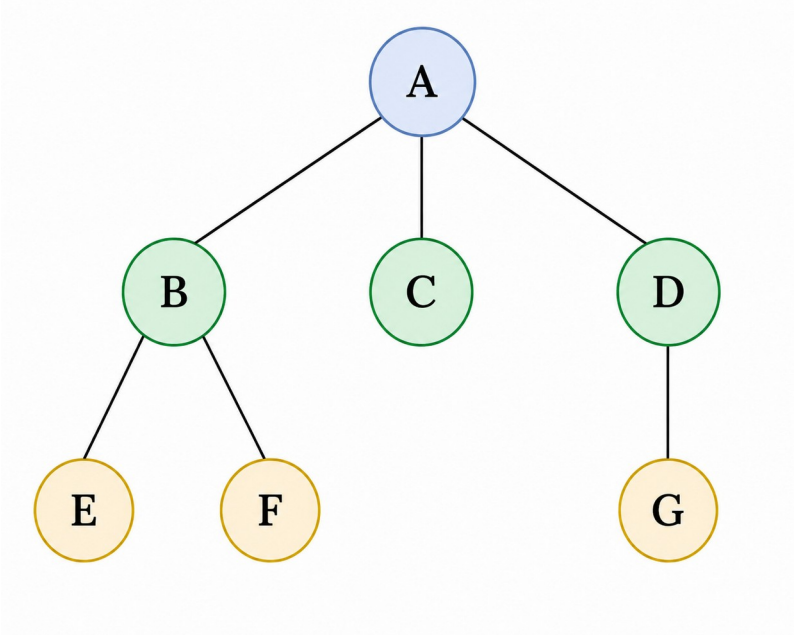
Dasharatha has 4 children. Each of them have their own children. Clearly this will not be in linear.

We do require non-linear data structures which allows one to many relatioinship.

In this course, we will study trees and graphs.

Tree:

A **tree** is a non-linear data structure used to represent hierarchical relationships. Unlike arrays, stacks, queues, and linked lists, where elements are arranged sequentially, a tree organizes elements in a parent–child structure.



This is an example of a tree.

Terminology from genealogy:

We say that A is the parent. B, C, D are children of A. They may be unordered or ordered left to right. B has two children E and F. C has no children. D has a single child G.

We say that A is the grand parent of E, F and G. We say that E, F, G are grand children of A.

B, C, D are siblings – children of the same parent.

A is an ancestor of B and also E. B and E are descendants of A.

C is the uncle of E – sibling of the parent. If there is ordering, then then C is the right uncle of E!

Sometimes these get coloured. We then talk about red uncle and black uncle.

We avoid terms indicating gender- no daughter, no mother. But so far, we are not able to find a gender-neutral term for the sibling of the parent!

Terminology from Botany:

All are referred to as nodes.

We say that A is the root. E, F, G are leaves. They are also called external nodes.

A, B, C, D are called internal nodes. The link between a parent and its child is called a branch.

A path is a sequence of nodes connected by edges.

A tree is recursively defined.

It is either empty Or

It has a root followed by number of trees – these trees are called subtree.

Summary of Terminologies:

A tree consists of a set of nodes connected by edges.

Root: the topmost node of the tree.

Parent: a node that has one or more children.

Child: a node directly below another node.

Siblings: nodes having the same parent.

Leaf / External node: a node with no children.

Internal node: a node having at least one child.

Edge: connection between a parent and a child.

Path: sequence of nodes connected by edges.

Ancestor: a node occurring on the path from the root to a given node.

Descendant: a node occurring below a given node.

Subtree: a tree formed by a node together with all its descendants.

A few more terminologies:

- The **degree of a node** is the number of children it has.

In the example above:

degree(A) = 3

degree(B) = 2

degree(C) = 0

degree(D) = 1

The **degree of a tree** is the maximum degree of any node in the tree.

Therefore, the degree of the above tree is 3.

- **Depth and level**

The **depth of a node** is the number of edges from the root to that node.

Depth(A) = 0

Depth(B) = 1

Depth(E) = 2

Nodes having the same depth are said to be at the same **level**.

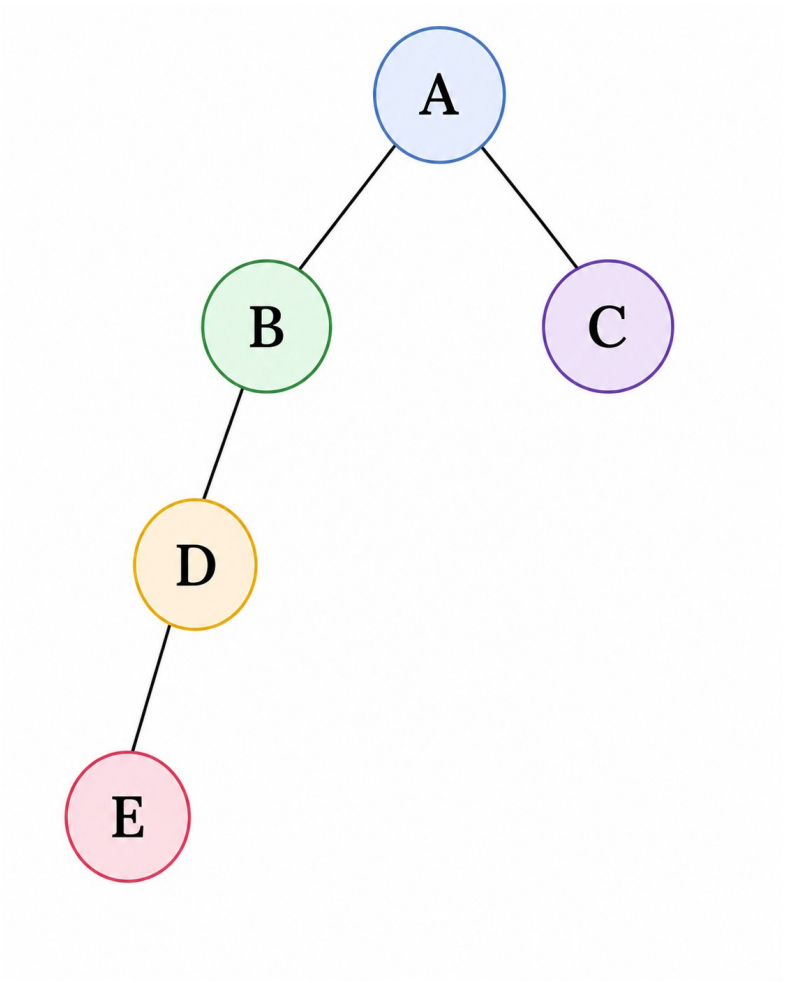
- **Height**

The **height of a node** is the length of the longest path from that node to a leaf.

A leaf has height 0.

The **height of a tree** is the height of its root.

For example:



The longest root-to-leaf path is:

A → B → D → E

It contains 3 edges. Therefore,

Height of tree = 3

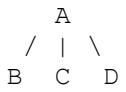
Important properties of trees:

- Acyclic : Can not start from a node, traverse and come back to the same node
- Connected. Any two nodes are connected by a path which may go through the root
- A tree of n nodes has n – 1 edges – neither more not less.
- Add an edge, we will have a cycle. Remove an edge, we will have more than one tree.

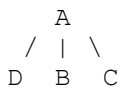
Ordered and unordered trees

In an **ordered tree**, the order of the children matters.

For example,



is considered different from:



if the tree is ordered.

In an **unordered tree**, only the parent–child relationships matter.

Forest

A **forest** is a collection of disjoint trees.

Removing the root of a tree produces a forest containing the subtrees rooted at the children of the original root.

A table indicating all the properties of the tree shown above.

Node information

Node	Parent	Children	Degree	Type
A	—	*B, C, D*	*3*	*Internal*
B	*A*	*E, F*	*2*	*Internal*
C	*A*	—	*0*	*Leaf*
D	*A*	*G*	*1*	*Internal*
E	*B*	—	*0*	*Leaf*
F	*B*	—	*0*	*Leaf*
G	*D*	—	*0*	*Leaf*

Important properties

- **Number of nodes:** *7*
- **Number of edges:** *6*
- **Root:** *A*
- **Leaves:** *C, E, F, G*
- **Internal nodes:** *A, B, D*
- **Degree of tree:** *3*
- **Height:** *3 edges*
- **Depth of A:** *0*
- **Depth of B, C, D:** *1*
- **Depth of E, F, G:** *2*

A possible ADT for a general tree :

- Create a tree
- Destroy the tree
- Check empty
- Check full
- Add a node
- Remove a node
- Traverse a tree
- Find root
- Find children
- Check whether leaf
- Find height of the tree
- Find the path from the root to a given node

…

In this course, we will discuss at length, a variation of trees called binary tree.

Application of Trees:

- **File System Structure:**

  This is hierarchical. A folder or a directory can have folders and directories within them apart from other files.
- **Search Trees:**

  To make searching efficient when the data is in memory, these trees are used.
- **Databases, indexing and B-trees:**

  To make searching more efficient when the data cannot be fit in memory
- **Expression Evaluation:**

  We discussed conversion of infix to postfix and evaluation of expression in postfix. Expression tree is another way of evaluation of expressions.
- **Compiler design:**

  While parsing a program, the compiler creates a tree called abstract syntax tree.
- **Document representation**

  Both XML and JSON allow for nesting forming effectively tree structure
- **Heap Tree and priority queue:**

  This is the most efficient way of implementing a priority queue
- **Process structure:**

  Processes in an operating system have relationship of the parent and the child.
- Organization structure
- Decision Trees in AI

  …

Binary Tree:

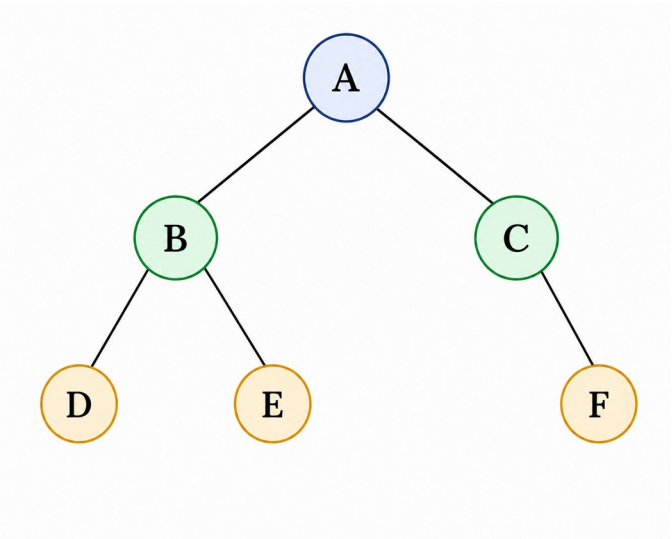
A **binary tree** is a tree in which every node has **at most two children**. The two children are distinguished as the **left child** and **right child**.

A binary tree is either:

- An **empty tree**, or
- A root node together with two disjoint binary trees called its:
  - **left subtree**
  - **right subtree**

Please observe the recursive nature of this definition. As the data structure is inherently recursive, writing recursive functions to support the interface is easy.

An example :



- A is the root.
- B is the left child of A.
- C is the right child of A.
- D and E are children of B.
- F is the right child of C.
- D, E, and F are leaves.

Binary Tree : Abstract Data Type

These are possible operations on a binary tree.

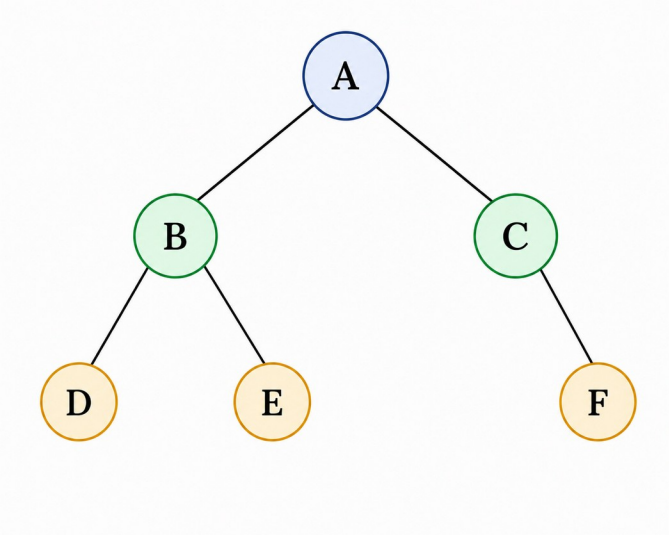
Create()
Create an empty tree
Destroy()
Remove the tree
root()
Return the root of the tree.
parent(p)
Return the parent of node p.

left(p)
Return the left child of p.
right(p)
Return the right child of p.
isLeaf(p)
Test whether p is a leaf.
component(p)
Return the component stored at p.
addRoot(x)
Create a root containing x.
addLeft(p, x)
Add x as the left child of p.
addRight(p, x)
Add x as the right child of p.
remove(p)
Remove node p and its subtree.
size()
Return the number of nodes.
Is_empty()
Test whether the tree is empty.
Is_full()
Check whether a component can be added
height()
Return the height of the tree.

We may implement a few of these interfaces in the next sections.

Inorder traversal of a binary tree:

We can visit all the components in an array or a list from one end to another. It is not possible to do so in case of a tree. One of the ways of visiting all the components in a binary tree is to visit the left subtree somehow and then the root and then the right subtree somehow. If the tree is empty, there is nothing to visit. As the left and the right subtrees are trees, we could apply this logic recursively.



Before visiting A, we visit left sub tree of A – that is the subtree B.

Before visiting B, we visit left sub tree of B – that is the subtree D.

Left sub tree of D is empty. So **visit D**.

Right sub tree of D is empty. So the sub tree D has been visited.

Left sub tree of B has been visited. So visit B.

Now visit the right sub tree of B.

Next we **visit E.**

Now, the left sub tree of A is visited. So **visit A** and then its right subtree.

This results in **visit C** and then **visit F.**

So the traversal of this tree will result in :

D B E A C F

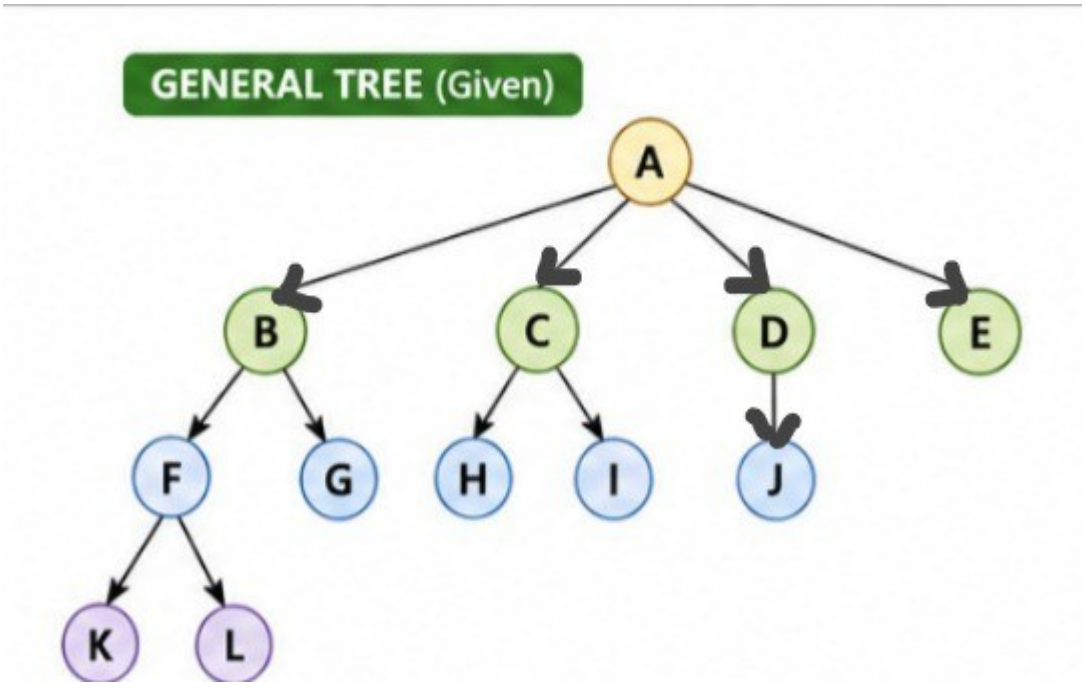
Conversion of any tree to a binary tree:

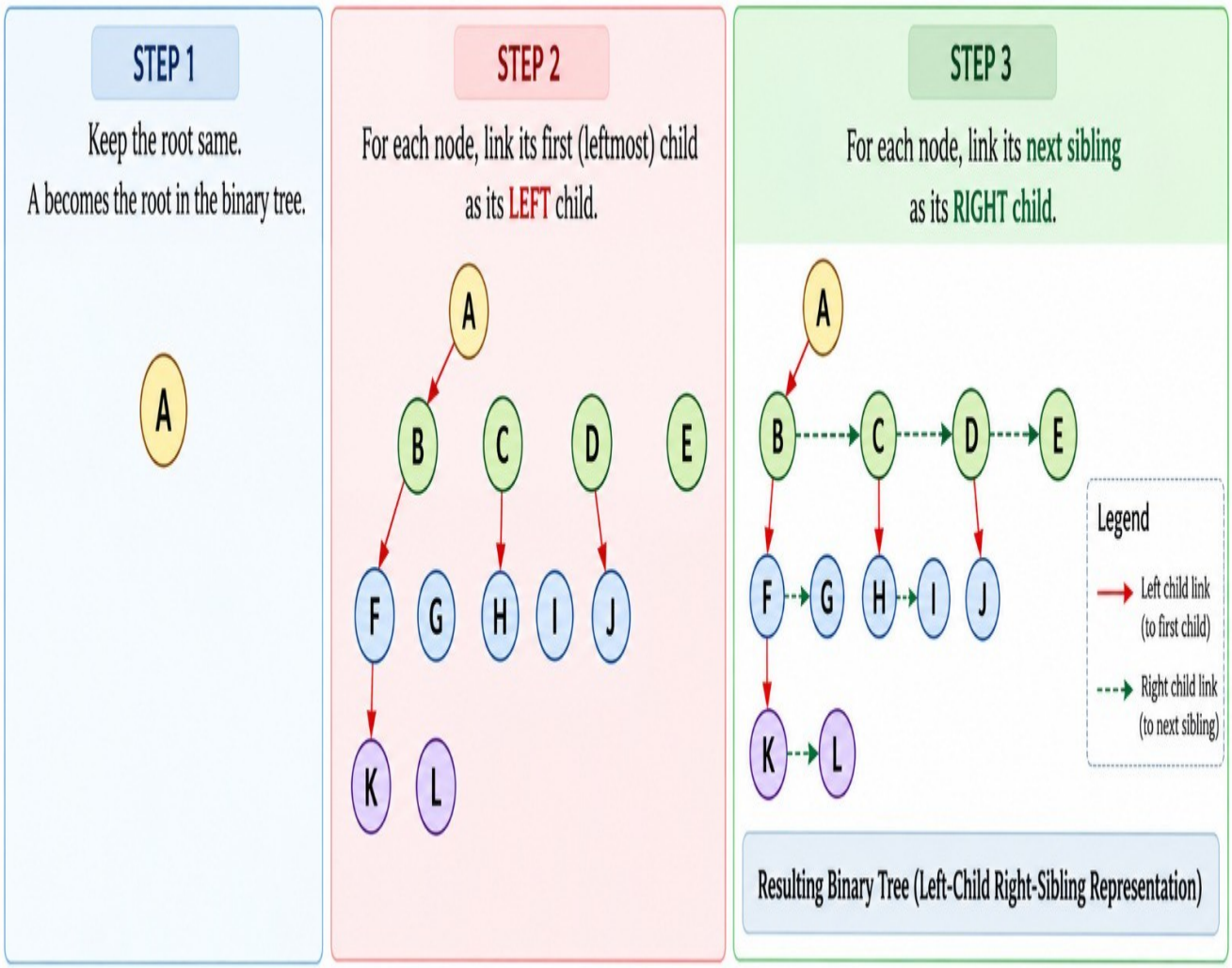
A general tree might have nodes with any number of children. Processing such a tree is difficult. But, if we can represent the general tree as a binary tree, then processing becomes easy.

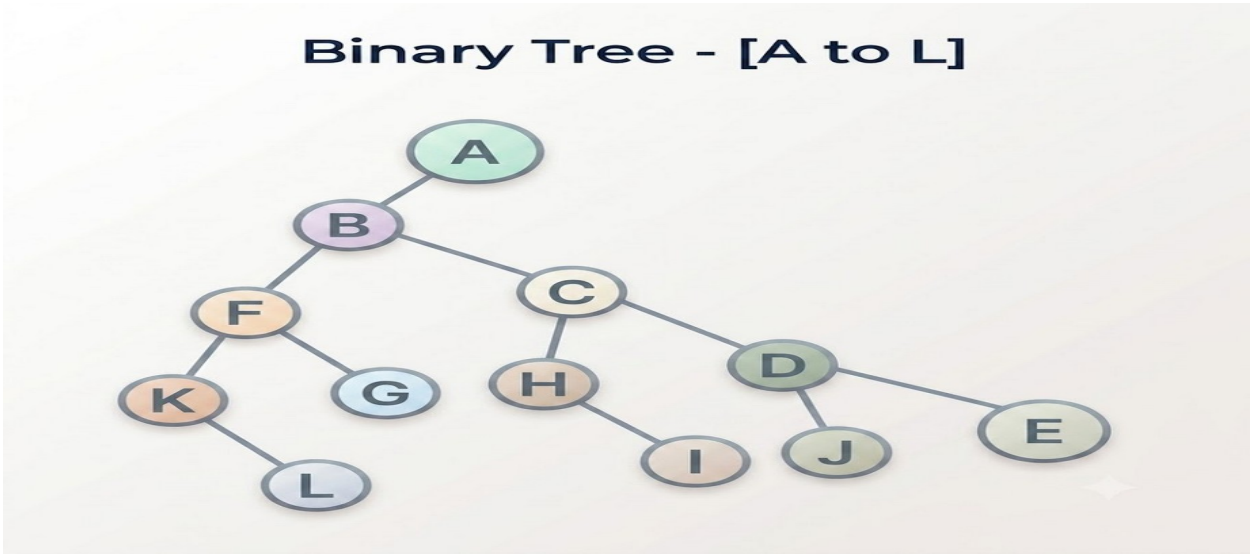
A binary tree can represent only two properties – one for the left branch and one for the right branch. If we can match this to two properties of a general tree, then we can represent any general tree as a binary tree.

This is the way we do it.

- Root of general tree will become the root of the binary tree
- Child of the node in a general tree becomes the left child of that node in the binary tree
- Sibling a node in a general tree becomes the right child of that node or attached to the right path.




STEP 1
Keep the root same.
A becomes the root in the binary tree.
A
STEP 2
For each node, link its first (leftmost) child as its LEFT child.
A
B
C
D
E
F
G
H
I
J
K
L
STEP 3
For each node, link its next sibling as its RIGHT child.
A
B
C
D
E
F
G
H
I
J
K
L
Legend
Left child link (to first child)
Right child link (to next sibling)
Resulting Binary Tree (Left-Child Right-Sibling Representation)


Binary Tree - [A to L]
A
B
C
F
D
K
G
H
E
I
J
L

Implementation of Binary Search Tree:

To start with, we will make a simple example of binary trees. We will use a special binary tree called binary search tree.

In a binary search tree, every component has a key. The components are arranged based on the key.

If the tree is empty, the component is put at the root.

Otherwise, if the key in the component at the root is higher than the key in the new component, we will insert in the left subtree.

Otherwise, we will in the right subtree.

You will observe that the insertion always happens as a leaf.

One of the implementations will be as follows.

The tree has a root. This like a list having a head.

The root points to a node structure.

The node structure has a component – called key – in this example.

Apart from that, it has two pointers – which point to the left and the right sub trees.

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;

struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;
```

This is the simplest interface to start experimenting.

void init(tree_t *ptr_tree);

- Ground the root.

void deinit(tree_t *ptr_tree);

- Walk through the tree
- Remove the left subtree
- Remove the right subtree
- Remove the root

void disp(tree_t *ptr_tree);

- If the tree is not empty
    - Display the left subtree
    - Display the root
    - Display the right subtree

void insert(tree_t *ptr_tree, int key);

- If the tree is empty make this the root
- Otherwise compare the keys
- Decide to insert in the left subtree or the right subtree

We mentioned that the tree is inherently a recursive data structure.

But the tree structure by itself is not recursive – the node structure which is self-referential is recursive. So, the functions in the tree call their corresponding recursive counter part in nodes.

Let us have a look at them.

Traversal:

Observe that the tree function disp calls disp_recursive of the node.

The disp_function of the node calls itself on its left subtree, displays the root and

Calls itself on its right subtree.

```
void disp(tree_t *ptr_tree)
{
	printf("Tree : ");
	disp_recursive(ptr_tree->root_);
	printf("\n");
}

static void disp_recursive(node_t* temp)
{
	if(temp)
	{
		disp_recursive(temp->left_);
		printf("%d ", temp->key_);
		disp_recursive(temp->right_);
	}
}
```

Insert:

```
void insert(tree_t *ptr_tree, int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        ptr_tree->root_ = insert_recursive(ptr_tree->root_, temp);
}
```

The above tree routine creates a node. Assumes that it succeeds in memory allocation. Populates the node. Calls the insert_recursive function of the node to return the root.

This function is very interesting. If the tree is empty, then this new node becomes the root. Otherwise based on the comparison of the key, we try to insert in the left subtree or the right subtree. If the insertion is in the left subtree, then left of the root is the root for that subtree. Similarly, if the insertion is in the right subtree, the right of the root is the root for that subtree. So, the return value of the function is assigned to the appropriate root.

You may want to draw and check how this works for a tree of at least 3 nodes.

```
static node_t* insert_recursive(node_t* root, node_t* temp)
{
        if(root == NULL)
        {
                root = temp;
        }
        else if(root->key_ > temp->key_)
        {
                root->left_ = insert_recursive(root->left_, temp);
        }
```

```
        else
        {
                root->right_ = insert_recursive(root->right_, temp);
        }
        return root;
}
```

Deinitialize:

Like other functions, deinit of tree calls the deinit_recursive of node.

```
void deinit(tree_t *ptr_tree)
{
        deinit_recursive(ptr_tree->root_);
        ptr_tree->root_ = NULL;
}
```

This recursive function on the node clears the left subtree recursively, then the right subtree recursively and then the root.

Please node that the order matters.

```
static void deinit_recursive(node_t* root)
{
        if(root)
        {
                deinit_recursive(root->left_);
                deinit_recursive(root->right_);
                free(root);
        }
}
```

Adding a few more functions in the interface of the tree:

Count the number of nodes

int number_of_nodes(tree_t *ptr_tree);

This function of tree calls the recursive function of the node.

The number of nodes in an empty tree is 0.

Otherwise we count the root node and add the number of nodes in the left subtree and the right subtree.

```
// tree
int number_of_nodes(tree_t *ptr_tree)
{
	return number_of_nodes_recursive(ptr_tree->root_);
}
int number_of_nodes_recursive(node_t* temp)
{
	if(temp == NULL)
	{
		return 0;
	}
	else
	{
		return 1 + number_of_nodes_recursive(temp->left_)
			+ number_of_nodes_recursive(temp->right_);
	}
}
```

Count the number of leaves

int number_of_leaves(tree_t *ptr_tree);

This function of tree calls the recursive function of the node.

The number of leaves in an empty tree is 0.

If both the left and right subtrees are empty, it is a leaf. We count it.

Otherwise we count the number of leaves in the left subtree and the number of leaves in the right subtree.

```
// node

int number_of_leaves(tree_t *ptr_tree)
{
	return number_of_leaves_recursive(ptr_tree->root_);
}
int number_of_leaves_recursive(node_t* temp)
{
	if(temp == NULL)
	{
		return 0;
	}
	else if(temp->left_ == NULL && temp->right_ == NULL)
	{
		return 1;
	}
	else
	{
		return number_of_leaves_recursive(temp->left_)
			+ number_of_leaves_recursive(temp->right_);
	}
}
```

deque

A few points:

Deque is a data structure which stands for **double ended queue.** It is pronounced as **deck.**

This data structure has two ends and at each end, supports addition as well as deletion.

It is normally implemented as a doubly linked list. Otherwise at least one of the 4 operations would be inefficient requiring as many operations as the number of elements in the data structure.

In fact, the doubly linked list that we implemented with limited interface is a deque.

Once a data structure has been implemented, we do not directly modify it. We extend it by either inheritance in an object oriented language or by composition. As 'C' is not an object oriented language, we will limit our discussion to extending the use of data structure by composition.

We shall implement a stack using deque. We do not want to reinvent wheels. If already a data structure provides the required interface and implementation, we match out interface to that of the existing data structure.

Stack implementation using deque by composition :

This is the required stack interface.

```
void push(stack_t* ptr_stack, int key);

int pop(stack_t* ptr_stack);

int is_empty_stack(stack_t* ptr_stack);

int is_full_stack(stack_t* ptr_stack);



void init_stack(stack_t* ptr_stack);

void deinit_stack(stack_t* ptr_stack);
```

our stack data structure shall embed a deque implementation – in our example dll_t.

```
struct stack
{
        dll_t list_; // embedding or layering or containment or composition
};
typedef struct stack stack_t;
```

All our stack functions would invoke functions on the embedded object.

```
void push(stack_t* ptr_stack, int key)
{
	add_in_begin(&ptr_stack->list_, key);
}
int pop(stack_t* ptr_stack)
{
	return remove_in_begin(&ptr_stack->list_);
}
int is_empty_stack(stack_t* ptr_stack)
{
	return is_empty(&ptr_stack->list_);
}
int is_full_stack(stack_t* ptr_stack)
{
	return is_full(&ptr_stack->list_);
}
```

```
void init_stack(stack_t* ptr_stack)
{
	init(&ptr_stack->list_); // delegation
}
void deinit_stack(stack_t* ptr_stack)
{
	deinit(&ptr_stack->list_);
}
```

The functions of stack in turn delegate the work to our deque dll.c.

In C++ library, there are 4 primary data structures – array, vector, deque and list. The data structures like stack and queue are considered secondary data structures. These are implemented by embedding one of these primary data structures.